

DSWoK: Нейронные сети и обработка естественного языка

Учебное пособие для самостоятельного изучения

Перевод и адаптация открытой базы знаний
DSWoK — Data Science Well of Knowledge
github.com/Erlemar/dswok

Содержание

Раздел 1. Внимание и стратегии негативной выборки

1.1 Механизм внимания (Attention)

- 1.1.1 Self-Attention
- 1.1.2 Cross-Attention
- 1.1.3 Multi-Head Attention
- 1.1.4 MultiQuery Attention (MQA)
- 1.1.5 Grouped-Query Attention (GQA)
- 1.1.6 Global vs. Local Attention
- 1.1.7 Multi-token attention

1.2 Negative sampling

- 1.2.1 Зачем сэмплировать
- 1.2.2 Истоки
- 1.2.3 Применения
- 1.2.4 Стратегии сэмплирования
- 1.2.5 Random / uniform
- 1.2.6 На основе популярности
- 1.2.7 In-batch
- 1.2.8 Hard negatives
- 1.2.9 Exposure-based
- 1.2.10 Смешанные стратегии
- 1.2.11 Сводка компромиссов
- 1.2.12 Смещения и ложные негативы
- 1.2.13 Ложные негативы
- 1.2.14 Случайные совпадения (accidental hits)
- 1.2.15 Калибровка
- 1.2.16 Формулировки функций потерь
- 1.2.17 Практические соображения
- 1.2.18 Ограничения
- 1.2.19 Пример кода
- 1.2.20 Ссылки

Раздел 2. Глубокие архитектуры рекомендательных систем, адаптация и мультимодальные модели

2.1 Двухбашенная модель (Two-tower)

- Архитектура и её варианты
- Входные данные и их подготовка
- Обучение
- Inference и serving
- Практические соображения

2.2 Wide & Deep Learning

- Архитектура
- Преимущества
- Недостатки
- Возможные улучшения
- Ссылки

2.3 Deep & Cross Network

- Архитектура
- Улучшения DCN v2
- Преимущества
- Недостатки
- Ссылки

2.4 DLRM (Deep Learning Recommendation Model)

- Архитектура
- Преимущества
- Недостатки
- Ссылки

2.5 Neural Collaborative Filtering (NCF)

- Архитектура
- Преимущества
- Недостатки
- Возможные улучшения

Ссылки

2.6 logQ-коррекция

Вычисление $Q(j)$

Ссылки

2.7 LoRA (Low-Rank Adaptation)

Основная идея

Преимущества

Ключевые гиперпараметры

Варианты

Ссылки

2.8 CLIP (Contrastive Language-Image Pre-training)

Архитектура

Обучение

Zero-shot классификация

Дообучение CLIP

Практические соображения

Ссылки

Раздел 3. Обработка естественного языка: базовые концепции и эмбединги

3.1 Введение в NLP (обзор задач и пайплайна)

3.2 Эмбединги слов (общая идея)

3.3 Word2Vec

3.4 GloVe

3.5 fastText

3.6 TF-IDF (Term Frequency-Inverse Document Frequency)

3.7 Рекуррентные нейросети (RNN)

Раздел 4. Трансформер, BERT и Retrieval-Augmented Generation

4.1 Архитектура Transformer

Vanilla Transformer

Вариации

Стратегии декодирования

Известные архитектуры

Дополнительная информация

4.2 BERT

Варианты и расширения

Ссылки

4.3 Retrieval-Augmented Generation (RAG)

Indexing (офлайн-подготовка данных)

Retrieval and Generation (онлайн / рантайм-обработка запроса)

Дополнительные улучшения и вариации

Оценка качества (Evaluation)

Раздел 5. Тематическое моделирование

5.1 Тематическое моделирование: общая постановка задачи

5.1.1 Ключевые допущения

5.1.2 Когда тематическое моделирование не подходит

5.1.3 Когда тематическое моделирование всё ещё уместно

5.1.4 Оценка качества

5.1.5 Выбор числа тем

5.1.6 Предобработка

5.1.7 Ссылки

5.2 Latent Dirichlet Allocation (LDA)

5.2.1 Порождающий процесс

5.2.2 Инференция

5.2.3 Гиперпараметры

5.2.4 Почему счётчики, а не TF-IDF

5.2.5 Индуктивный инференс на новых документах

5.2.6 Управляемые и supervised-варианты

5.2.7 Достоинства и недостатки

5.2.8 Когда LDA всё ещё уместен

5.2.9 Ссылки

5.3 Методы тематического моделирования (обзор семейств методов)

5.3.1 Классические методы

5.3.2 Нейросетевые тематические модели

5.3.3 Методы на основе эмбедингов

5.3.4 Обнаружение тем с помощью LLM

5.3.5 Варианты для коротких текстов

5.3.6 Другие заметные подходы

5.3.7 Когда что использовать

5.3.8 Инструменты и библиотеки

5.3.9 Ссылки

5.4 BERTopic

5.4.1 Пайплайн

5.4.2 c-TF-IDF

5.4.3 Ключевые возможности

5.4.4 Режимы отказа

5.4.5 Индуктивный инференс на новых документах

5.4.6 Смешанное членство

5.4.7 Guided BERTopic

5.4.8 Достоинства и недостатки

5.4.9 Ссылки

Нейронные сети и обработка естественного языка

Раздел 1. Внимание и стратегии негативной выборки

1.1 Механизм внимания (Attention)

Оригинальная статья

Внимание (attention) — механизм, позволяющий нейронным сетям фокусироваться на конкретных частях входной последовательности. Этот механизм исследовался в различных архитектурах, таких как Hyena и Linformer, для повышения вычислительной эффективности. Внимание также широко применяется в компьютерном зрении: например, в Swin Transformers и моделях детекции объектов вроде DETR.

Фундаментальный вариант — Scaled Dot-Product Attention (используется в «Transformer»). У него три входа: - Query (Q): текущий токен, который собирает информацию. - Key (K): представление каждого токена последовательности, доступного для внимания. - Value (V): то, что предоставляет каждый токен, если он выбран механизмом внимания.

Пошаговое вычисление внимания: 1. Измеряем, насколько каждый ключ K_i релевантен query Q , через скалярное произведение: $\text{scores} = Q \times K^T$ 2. Чтобы сохранить устойчивость значений при больших эмбедингах, делим на $\sqrt{d_k}$, где d_k — размерность векторов ключей: $\text{scaled_scores} = \frac{Q \times K^T}{\sqrt{d_k}}$. Когда d_k велико, скалярное произведение может стать большим по модулю, выталкивая softmax в области с очень малыми градиентами. Дисперсия скалярного произведения растёт с d_k . Поэтому используется именно квадратный корень — мы нормируем масштаб. 3. Преобразуем оценки в распределение вероятностей, чтобы понять, сколько внимания уделить каждому элементу: $\alpha = \text{softmax}(\text{scaled_scores})$. 4. Умножаем каждое значение V_i на его вес внимания α_i и суммируем, получая итоговый выход: $\text{Attention}(Q, K, V) = \alpha \times V$ В результате получается контекстный вектор, который выделяет наиболее релевантную информацию из V для query Q .

Кратко: внимание вычисляет взвешенную сумму входных элементов (values), причём веса определяются функцией совместимости между query и соответствующими keys: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$

[!hint] Представьте, что вы на большой вечеринке и пытаетесь сосредоточиться на конкретном разговоре. Про каждого человека вы спрашиваете себя: «Насколько то, что говорит этот человек, релевантно тому, что я хочу узнать?» (вычисление оценок внимания). Затем вы больше фокусируетесь на тех, кто даёт полезную информацию (применение весов внимания), сохраняя при этом некоторую осведомлённость обо всех остальных. Ваш мозг объединяет всю эту информацию, придавая больший вес важным источникам (взвешенная сумма values).

[!hint] Простое объяснение: внимание — это словарь с аппроксимацией. В обычном словаре есть пары ключ-значение, и мы передаём запрос, чтобы получить результат. Мы либо получаем значение ключа, либо ничего. В attention мы получаем ответ, даже если не можем найти точный ключ.

1.1.1 Self-Attention

- Keys, queries и values берутся из одной и той же исходной последовательности.
- Каждая позиция может уделять внимание всем позициям последовательности.

1.1.2 Cross-Attention

- Queries берутся из одной последовательности (например, декодер в seq2seq-модели), а keys и values — из другой (например, энкодер).
- Часто используется в машинном переводе и генеративных задачах, где одна последовательность уделяет внимание другой.

1.1.3 Multi-Head Attention

- Выполняет несколько механизмов внимания параллельно.
- Позволяет модели одновременно уделять внимание информации из разных подпространств представлений на разных позициях. Каждая голова потенциально может научиться фокусироваться на разных типах связей или признаков.

1.1.4 MultiQuery Attention (MQA)

- Все query-головы используют общие матрицы keys и values; различаются только матрицы queries.
- Существенно снижает требования к памяти и время инференса.
- Может приводить к снижению качества по сравнению с MHA.

1.1.5 Grouped-Query Attention (GQA)

- Введён для баланса между эффективностью MQA и качеством MHA.
- В MHA у каждой query-головы свои key-value-головы (максимум качества, но высокое потребление памяти). В MQA все query-головы используют одну общую key-value-голову (максимум эффективности, но ниже качество). GQA делит query-головы на группы, где каждая группа использует общий набор key-value-голов.
- Число групп (G) — гиперпараметр: больше групп — ближе к MHA, меньше — ближе к MQA.
- Применяется в моделях Llama 2-70B, Mistral 7B, Falcon 40B. Особенно полезен в multi-GPU-окружениях с тензорным параллелизмом.

1.1.6 Global vs. Local Attention

- Global Attention уделяет внимание всем позициям последовательности (стандартный подход). Помогает сохранять дальние зависимости, которые local attention может упустить.
- Local Attention уделяет внимание только окну позиций вокруг текущей позиции. Снижает вычислительную сложность с $O(n^2)$ до $O(n)$.
- Архитектуры вроде Longformer и BigBird используют гибридные подходы, сочетающие оба варианта: local attention для большинства токенов плюс некоторая форма global attention (отдельные токены, которые работают глобально, или разреженные глобальные паттерны), чтобы сохранить способность ловить дальние зависимости там, где это нужно.

1.1.7 Multi-token attention

- Устраняет ограничения single-token attention, где отдельные веса определяются сходством только одной пары query-key.
- Применяет операции свёртки к queries, keys и головам, позволяя соседним токенам влиять на веса внимания друг друга.

[example]- Код Scaled dot attention ``python import torch import torch.nn.functional as F

```
def scaled_dot_product_attention(Q, K, V, mask=None): """ Q, K, V: (batch_size, seq_len, dim) mask (batch_size, seq_len, seq_len) to prevent attention on certain positions """ d_k = Q.size(-1) # dimensionality scores = torch.matmul(Q, K.transpose(-2, -1)) # (batch_size, seq_len, seq_len) scores = scores / (d_k ** 0.5) # scale
```

```
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))

    # softmax along the last dimension
    attn_weights = F.softmax(scores, dim=-1) # (batch_size, seq_len, seq_len)

    # multiply by values
    output = torch.matmul(attn_weights, V) # (batch_size, seq_len, dim)

    return output, attn_weights
```

...

[example]- Код Multi-head self-attention ``python class SelfAttention(nn.Module): """ This class implements multi-head self-attention. """ def init(self, embed_size, heads): super(SelfAttention, self).init() self.embed_size = embed_size self.heads = heads self.head_dim = embed_size // heads

```
    assert (self.head_dim * heads == embed_size), "Embed size must be divisible by heads"

    # Linear projections
    self.q_linear = nn.Linear(embed_size, embed_size)
    self.k_linear = nn.Linear(embed_size, embed_size)
    self.v_linear = nn.Linear(embed_size, embed_size)
    self.out_linear = nn.Linear(embed_size, embed_size)

    def forward(self, q, k, v, mask=None):
        batch_size = q.size(0)

        # Linear projections and reshape for multi-head
        q = self.q_linear(q).view(batch_size, -1, self.heads, self.head_dim).permute(0, 2, 1, 3)
        k = self.k_linear(k).view(batch_size, -1, self.heads, self.head_dim).permute(0, 2, 1, 3)
        v = self.v_linear(v).view(batch_size, -1, self.heads, self.head_dim).permute(0, 2, 1, 3)

        # Compute attention scores
        scores = torch.matmul(q, k.transpose(-2, -1)) / (self.head_dim ** 0.5)

        # Apply mask (if provided)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        # Normalize scores to probabilities
        attention_weights = torch.softmax(scores, dim=-1)

        # Compute weighted sum
        out = torch.matmul(attention_weights, v)

        # Reshape and apply final linear projection
        out = out.permute(0, 2, 1, 3).contiguous().view(batch_size, -1, self.embed_size)
        out = self.out_linear(out)

        return out, attention_weights
```

...

1.2 Negative sampling

Negative sampling обучает модель, противопоставляя каждому наблюдаемому положительному примеру небольшой набор сэмплованных альтернатив вместо всех элементов каталога. Для одних функций потерь это аппроксимирует полный softmax или попарную целевую функцию; для других (SGNS, BCE с сэмплованными негативами) сам сэмплованный набор и определяет целевую функцию напрямую. Подход удерживает стоимость обучения на приемлемом уровне для каталогов с миллионами элементов.

Технику популяризовали Mikolov et al. (2013) для Word2Vec как упрощённый вариант noise contrastive estimation (NCE, Gutmann & Hyvärinen 2010); контрастивное оценивание относительно шумового распределения появилось раньше. Применяется широко: эмбединги в NLP, «рекомендательные системы», retrieval в «двухбашенных моделях», contrastive обучение представлений, эмбединги графов знаний.

1.2.1 Зачем сэмплировать

Полный softmax по миллионам элементов стоит $O(|\text{catalog}|)$ за один шаг обучения. Системы с implicit feedback вдобавок наблюдают только положительные взаимодействия (клики, проигрывания, покупки). Если считать каждую пару без взаимодействия отрицательной, это раздувает шум разметки; большинство получаемых градиентов несёт мало сигнала.

Сэмплованные отрицательные примеры дают контрастивный сигнал ценой доли вычислений. Для положительной пары (u, i^+) , оцениваемой относительно K сэмплованных отрицательных i_k^- , взятых равномерно, форма sampled-softmax выглядит так:

$$\mathcal{L} = -\log \frac{\exp(s_{u,i^+})}{\exp(s_{u,i^+}) + \sum_{k=1}^K \exp(s_{u,i_k^-})}$$

Когда форма sampled-softmax должна аппроксимировать полный softmax при неравномерном распределении сэмплования Q , нескорректированная оценка смещена: у элементов, часто попадающих в негативы, оценки систематически занижаются. Поправка importance sampling (Bengio & Senécal 2008; стандарт в retrieval «двухбашенных моделей» после Yi et al. 2019) вычитает $\log Q$ из каждого логита:

$$\mathcal{L} = -\log \frac{\exp(s_{u,i^+} - \log Q(i^+))}{\exp(s_{u,i^+} - \log Q(i^+)) + \sum_{k=1}^K \exp(s_{u,i_k^-} - \log Q(i_k^-))}$$

Это и есть «logQ-коррекция». Pairwise BPR, BCE, SGNS и аналогичные бинарные или попарные целевые функции не аппроксимируют полный softmax и не нуждаются в той же коррекции; у них есть собственные вопросы калибровки.

Два следствия сэмплования, независимо от функции потерь: градиент становится оценкой с большей дисперсией, а распределение сэмплования формирует, какие именно контрасты экспонируются функции потерь.

1.2.2 Истоки

NCE переформулирует оценку плотности как задачу логистической регрессии относительно известного шумового распределения, где partition function — обучаемый параметр. Skip-Gram with Negative Sampling (SGNS) отбрасывает член partition function и обучает бинарный классификатор между наблюдаемыми парами (слово, контекст) и парами (слово, сэмплованный шум):

$$\mathcal{L}_{\text{SGNS}} = -\log \sum_{k=1}^K \sigma(s_{u,i_k^-}) - \sum_{k=1}^K \log \sigma(-s_{u,i_k^-})$$

Sampled-softmax и бинарный вариант SGNS разделяют идею negative sampling, но представляют собой разные целевые функции и дают разные градиенты. Многие крупномасштабные системы retrieval «двухбашенных моделей» используют целевые функции sampled-softmax; современное визуальное contrastive обучение использует InfoNCE; классический Word2Vec использовал SGNS. Mikolov брал отрицательные примеры из unigram-распределения в степени 3/4 и сообщает о лучшей точности на задачах с аналогиями, чем у unigram- и равномерных альтернатив.

Goldberg & Levy (2014) выводят целевую функцию SGNS. Levy & Goldberg (NeurIPS 2014) показывают, что она неявно факторизует сдвинутую матрицу pointwise mutual information (PMI) $\text{PMI}(w, c) = \log K$, где K — число сэмплованных отрицательных примеров; их количество напрямую управляет сдвигом.

1.2.3 Применения

Рецепт «сэмплируем и противопоставляем» встречается в самых разных сценариях обучения эмбедингов:

- Словесные эмбединги: SGNS для Word2Vec, исходное применение; отрицательные примеры — из unigram-распределения в степени 3/4.
- Коллаборативная фильтрация: «Neural Collaborative Filtering» использует сэмплованные негативы с binary cross-entropy; retriever'ы «двухбашенной модели» используют sampled softmax с «logQ-коррекцией».
- Визуальное и мультимодальное contrastive обучение: SimCLR и CLIP трактуют in-batch-альтернативы как негативы.
- Плотный текстовый retrieval: Dense Passage Retrieval обучает BERT-based dual encoders с офлайн-ANN-добытыми hard negatives из top-k предыдущей версии модели.
- Последовательные и сессионные рекомендации: SASRec, BERT4Rec и HSTU сэмплюют альтернативные следующие элементы как негативы; «негатив» — это правдоподобное-но-неверное продолжение сессии, а не несвязанный элемент.
- Metric learning и эмбединги графов знаний: triplet- и pairwise-функции потерь, где негативы формируют геометрию эмбедингов (TransE, DistMult и их преемники).

1.2.4 Стратегии сэмплования

Стратегии различаются преимущественно отрицательным распределением $Q(j)$.

1.2.5 Random / uniform

Негатив берётся равномерно из каталога. Дёшево и легко рассуждать о результате, но даёт в основном лёгкие отрицательные примеры, дающие слабые градиенты. На практике усиливает popularity bias (Prakash et al. 2024): положительные примеры сильно смещены в голову распределения, а равномерные негативы — в основном из хвоста, так что разграничение «голова против головы» получает мало обучающего сигнала.

1.2.6 На основе популярности

Негативы берутся пропорционально степени частоты элемента. Word2Vec использовал $Q(j) \propto f(j)^{0.75}$. Популярные элементы с большей вероятностью оказываются информативными негативами, потому что пользователь имел к ним экспозицию, но не взаимодействовал, что делает их правдоподобными контрпримерами. Когда это используется как аппроксимация sampled-softmax, требуется «logQ-коррекция», чтобы калибровка оценок оставалась верной.

1.2.7 In-batch

Внутри мини-батча из N пар (пользователь, положительный элемент) положительный элемент каждого пользователя служит негативом для всех остальных пользователей. Вычислительная стоимость пренебрежимо мала, а шаг даёт $N(N - 1)$ неявных негативов. In-batch-распределение отражает популярность в обучающих данных, поэтому популярные элементы перепредставлены среди негативов. Yi et al. (2019) ввели коррекцию смещения сэмплирования, ставшую стандартом для in-batch softmax в retrieval «двухбашенных моделей».

Иллюстрация демонстрирует проблему случайных совпадений (accidental hits): когда две записи батча имеют общий item ID (элемент A появляется и для пользователя 0, и для пользователя 3), внедиагональные ячейки, где элемент столбца равен положительному элементу другой строки, оказываются ложными негативами — это рассматривается в подразделе «Случайные совпадения» ниже.

1.2.8 Hard negatives

Негативы смещены в сторону элементов, которым модель сейчас даёт высокие оценки, но которые не являются положительными (Zhang et al. 2013). Hard negatives обостряют границу принятия решений и улучшают top-k-метрики. Типичные источники:

- На основе сходства: элементы с высоким сходством эмбедингов к положительному, но без положительного взаимодействия (часто та же категория, бренд или ценовой диапазон).
- На основе ранжирования: элементы, занявшие позиции чуть ниже положительного на первичном проходе retrieval; стандартный источник hard negatives в двухступенчатом retrieval — позиции 100–500 собственного вывода retriever'a.
- ANN-добытые: модель предыдущей версии строит индекс FAISS или ScaNN, извлекает top- N для каждого пользователя, исключает истинный положительный, а остальное замораживается как hard-набор для следующей эпохи. Стандарт в DPR-подобных текстовых retrieval; обновляется каждые несколько эпох.
- Cross-batch memory (Wang et al. 2020): очередь эмбедингов из предыдущих батчей, экспонирующая модели элементы, отсутствующие в текущем мини-батче.
- Momentum queue (He et al. 2020 для MoCo): медленно обновляемый momentum encoder поддерживает длинную FIFO-очередь прошлых эмбедингов как негативов, развязывая размер очереди от размера батча ценой небольшого смещения из-за устаревания.

Словарь metric learning (триплеты anchor / positive / negative, hard / semi-hard / easy negatives в смысле FaceNet) отображается на те же структурные выборы под другими названиями.

Компромиссы: обучение менее стабильно, ложные негативы сэмплируются чаще, а кандидатский набор нужно периодически обновлять, потому что то, что считается «сложным», смещается по ходу обучения. Добыча hard negatives из текущей модели может усиливать собственные смещения модели: элементы, которые модель уже неправильно ранжирует, склонны выбираться в hard negatives эпоху за эпохой, многократно наказывая неоднозначные положительные примеры. Слишком агрессивное «давление» может схлопнуть представления в узкий конус, где положительные и отрицательные оценки сходятся, а градиенты исчезают (коллапс эмбедингов). Shi et al. (2023) связывают Bayesian Personalized Ranking (BPR) с dynamic negative sampling и оптимизацией partial-AUC, увязывая выигрыш от hard negatives с производительностью top-k retrieval.

1.2.9 Exposure-based

Негатив ограничен элементами, которые пользователь действительно увидел, но не кликнул. Это «экспонированные не-положительные», а не подтверждённые негативы: позиция, внимание, UI-вёрстка и serving-политика — всё это вносит конфаундеры в отрицательный сигнал. Полезно как дополнение к другим стратегиям, редко достаточно само по себе; механика propensity, делающая exposure-based-негативы менее смещёнными, общая с «контрфактической оценкой».

1.2.10 Смешанные стратегии

Производственные retriever'ы обычно сочетают стратегии: база из in-batch или случайных негативов плюс небольшая доля hard negatives. Fan et al. (2023) сообщают о соотношении random к hard 100:1 с hard-элементами с ранговых позиций 100–500 и обнаруживают, что чисто hard-negative-обучение проигрывает только случайному сэмплированию. Смешение балансирует покрытие и сложность. Кёррикулярные расписания варьируют смесь во времени обучения, а не внутри каждого батча: начинают со случайных или взвешенных по популярности негативов, затем по эпохам наращивают долю hard negatives. Это стабилизирует обучение, когда чистое hard-negative-сэмплирование с первой эпохи слишком зашумлено.

1.2.11 Сводка компромиссов

Стратегия	Вычисления / память	Дисперсия градиента	Риск ложных негативов	Калибровка
Random / uniform	низкая	высокая	низкий	чистый сдвиг приора

Стратегия	Вычисления / память	Дисперсия градиента	Риск ложных негативов	Калибровка
Взвешенная по популярности	низкая	средняя	низкий–средний	неравномерное Q ; logQ-коррекция для sampled softmax
In-batch	пренебрежимая	средняя	средний (accidental hits)	logQ-коррекция для sampled softmax
ANN-добытые hard	высокая (периодическое построение индекса)	низкая	высокий	пере-калибровать на production-подобной валидации
Cross-batch memory	средняя (хранение очереди)	низкая	высокий	пере-калибровать
Momentum contrast	средняя–высокая (очередь + momentum encoder)	низкая	высокий	пере-калибровать
Exposure-based	средняя	средняя	низкий	наследует смещение serving-политики

1.2.12 Смещения и ложные негативы

Item popularity взаимодействует с сэмплированием несколькими способами. Положительные взаимодействия смещены в голову распределения, потому что популярные элементы получают больше экспозиции; случайное negative sampling тянет в основном хвостовые элементы, оставляя разграничение «голова против головы» недоученным. Сэмплирование, взвешенное по популярности, сдвигает массу к голове в качестве негативов, ценой необходимости «logQ-коррекции» для целевых функций sampled-softmax. In-batch-сэмплирование следует распределению положительных и наследует их «головную» тяжесть.

1.2.13 Ложные негативы

Implicit feedback содержит истинные негативы (подлинное отсутствие интереса) и ложные негативы (релевантные элементы, которых пользователь никогда не видел). Сэмплирование считает оба варианта негативами, смещая функцию потерь. Меры смягчения:

- Пропускать элементы, с которыми пользователь уже взаимодействовал (базовый, но обязательный шаг).
- Двухступенчатая фильтрация: candidate generation выдаёт релевантный набор, а сэмплированные негативы берутся из дополнения к нему.
- Фильтрация по дисперсии: истинные негативы сходятся к низким оценкам по ходу обучения; ложные негативы продолжают получать высокие. Элементы с низкой дисперсией loss после нескольких эпох можно помечать как кандидатов в ложные негативы (Ma et al. 2024).

1.2.14 Случайные совпадения (accidental hits)

При in-batch-сэмплировании две записи батча могут иметь общий item ID (пользователи A и B оба кликнули элемент X). Тогда матричное умножение наказывает пользователя A против копии элемента X у пользователя B как против негатива, гарантируя ложно-отрицательный loss на самых популярных элементах. Решение — маска равенства ID, применяемая к матрице логитов до softmax, с сохранением диагонали истинных положительных. Почти-дубликаты, варианты одного элемента, элементы от одного автора или товарного семейства и повторяющийся контент могут давать тот же тип ложных негативов даже при разных ID; в подходящих случаях маска расширяется до эквивалентности на основе сходства или семейства. После «logQ-коррекции» это наиболее значимая in-batch-коррекция.

1.2.15 Калибровка

Сэмплированные негативы меняют эффективную распространённость положительных в обучении. Модель, обученная при одном положительном на десять сэмплированных негативов, выдаёт вероятности, откалиброванные под распределение 1:10, а не под реальную частоту. Исправление — поправка приора. Сэмплирование, зависящее от признаков элемента (взвешенное по популярности, hard-negative), не является чистым сдвигом приора; пере-калибровка на production-подобной валидации — более безопасный путь. См. «Калибровку» для формальной поправки. (Pairwise BPR не подпадает под эту схему: его выход — оценка ранжирования, а не вероятность, так что калибровка к соотношению сэмплирования не применяется.)

1.2.16 Формулировки функций потерь

Negative sampling не зависит от выбранной функции потерь; одно и то же распределение сэмплирования можно применять к разным функциям потерь.

- Pairwise BPR (Rendle et al. 2009): $-\log \sigma(s_{u,i^+} - s_{u,i^-})$ по триплетам (пользователь, положительный, сэмплированный негатив). Напрямую оптимизирует попарную корректность ранжирования.
- Binary cross-entropy: каждая пара (пользователь, элемент) как 0/1-классификация с сэмплированными негативами. Стандарт в «Neural Collaborative Filtering».
- Sampled softmax: log-правдоподобие положительного против softmax по положительному плюс сэмплированным негативам. Стандарт в retrieval «двухбашенных моделей»; требует «logQ-коррекции», когда распределение сэмплирования смещено.
- InfoNCE / contrastive: softmax с температурным масштабированием и in-batch-альтернативами в качестве негативов (Chen et al. 2020 для SimCLR). Меньшая температура $\tau < 1$ заостряет softmax и концентрирует градиент на самых сложных негативах в батче; в целевых функциях InfoNCE-стиля температура может значить столько же, сколько и выбор распределения сэмплирования.

Выбор между pairwise и listwise зависит от оптимизируемой метрики. BPR оптимизирует попарную корректность ранжирования; sampled softmax нацелен на log-вероятность положительного внутри кандидатского набора и ближе соответствует целям top-k retrieval.

1.2.17 Практические соображения

- Числа SGNS: Mikolov 2013 рекомендует 5–20 негативов на положительный для небольших датасетов и 2–5 для крупных.
- Retrieval «двухбашенных моделей» обычно использует от нескольких сотен до нескольких тысяч in-batch-негативов (Yi et al. 2019 тестируют размеры батча до 8192).
- Сетапы InfoNCE и sampled-softmax должны L2-нормировать эмбединги перед скалярным произведением; иначе сеть может снижать loss, раздвывая модуль эмбедингов, а не выравнивая направление, и температура теряет задуманное значение.
- Стадии retrieval и ranking требуют разных распределений сэмплирования. Retrieval выигрывает от широкого покрытия (random + in-batch), чтобы найти границу релевантного/нерелевантного в масштабе каталога; ranking выигрывает от hard negatives из собственного top-k retriever'a, чтобы научиться различать среди уже правдоподобных кандидатов.
- Hard negatives нуждаются в графике обновления. Кэширование кандидатского набора через эпоху приводит к повторному использованию элементов, уже хорошо разделённых, и даёт мало градиентного сигнала.
- Для целевых функций sampled-softmax, призванных аппроксимировать полный softmax, применяйте «logQ-коррекцию», когда Q неравномерно; метрики ранжирования могут казаться неизменными, но калибровка оценок от этого зависит. BPR, BCE и SGNS обучают бинарные или попарные классификаторы напрямую и не нуждаются в той же поправке, хотя имеют собственные вопросы калибровки.
- Частота ложных негативов — полезный диагностический показатель. Внезапное падение NDCG на отложенном наборе после перехода на hard-negative-сэмплирование часто указывает на то, что сэмплированные положительные оцениваются как негативы.
- Распределение сэмплирования должно соответствовать политике candidate generation. Негативы, взятые далеко за пределами распределения candidate generation, несут мало градиентного сигнала; негативы внутри него несут риск ложных негативов.

flowchart LR

```
Q[Query / user] --> R["Retrieval<br>broad coverage<br>random + in-batch"]
R -->|top-k candidates| K["Ranking<br>hard negatives<br>from retriever top-k"]
K --> O[Final ranked list]
```

1.2.18 Ограничения

- Сэмплированный градиент — оценка с большей дисперсией, чем полный loss. При малом числе негативов на шаг дисперсия доминирует над шумом обучения; при большом — растёт время обучения.
- У оптимального распределения сэмплирования нет замкнутой формы. Обзоры устойчиво сообщают, что правильный выбор зависит от задачи и датасета (Ma et al. 2024).
- Модели, обученные с одним распределением сэмплирования, часто плохо переносятся на другое распределение во время инференса. Переключение между random и in-batch-сэмплированием между запусками обучения обычно требует пере-калибровки или полного дообучения.
- Гарантии состоятельности NCE опираются на известное шумовое распределение; в production подставляется оценка эмпирической частоты. Гарантии асимптотические, не выборочно-конечные.

1.2.19 Пример кода

[!example]- PyTorch BCE с равномерным negative sampling ``python import torch import torch.nn.functional as F

def negative_sampling_loss(user_emb, pos_item_emb, item_table, num_neg=4): # user_emb, pos_item_emb: (batch, dim) # item_table: (n_items, dim)
full item embedding matrix batch_size = user_emb.size(0) n_items = item_table.size(0)

```
# Uniform random negatives. Production code should reject candidates equal to the  
# positive item_id or in the user's positive history; this minimal example skips  
# that step for clarity.  
neg_idx = torch.randint(0, n_items, (batch_size, num_neg), device=user_emb.device)  
neg_emb = item_table[neg_idx] # (batch, num_neg, dim)  
  
pos_score = (user_emb * pos_item_emb).sum(-1) # (batch,)  
neg_score = (user_emb.unsqueeze(1) * neg_emb).sum(-1) # (batch, num_neg)  
  
pos_loss = F.binary_cross_entropy_with_logits(pos_score, torch.ones_like(pos_score))  
neg_loss = F.binary_cross_entropy_with_logits(neg_score, torch.zeros_like(neg_score))  
return pos_loss + neg_loss
```

`` Рецепт SGNS заменяет равномерный torch.randint на torch.multinomial по $f^{0.75}$.

[!example]- PyTorch in-batch sampled softmax с logQ-коррекцией и маскированием accidental hits ``python import torch import torch.nn.functional as F

def in_batch_sampled_softmax(user_emb, item_emb, item_ids, log_q, temperature=0.05): # user_emb, item_emb: (batch, dim) — paired positives,
diagonal of the score matrix # item_ids: (batch,) item IDs in the batch # log_q: (batch,) log probability of each item under the sampling distribution #
L2-normalize: otherwise the network can lower loss by inflating magnitudes user_emb = F.normalize(user_emb, dim=-1) item_emb =
F.normalize(item_emb, dim=-1)

```
logits = (user_emb @ item_emb.t()) / temperature # (batch, batch)
# correction applied in tempered space; (s - logQ)/τ ordering also exists in production
logits = logits - log_q.unsqueeze(0)

# Mask accidental hits: same item ID off-diagonal becomes -inf so it vanishes in softmax
id_mask = item_ids.unsqueeze(0) == item_ids.unsqueeze(1)
id_mask.fill_diagonal_(False)
logits = logits.masked_fill(id_mask, -1e9)

labels = torch.arange(user_emb.size(0), device=user_emb.device)
return F.cross_entropy(logits, labels)
```

``log\_q`` обычно оценивается из item-frequency-счётчиков, поддерживаемых параллельно с обучением. Без шага маскирования популярные элементы в батче молча штрафуются как негативы против собственных положительных в других строках.

1.2.20 Ссылки

- Mikolov, Sutskever, Chen, Corrado, Dean — *Distributed Representations of Words and Phrases and their Compositionality* (NIPS 2013)
- Goldberg & Levy — *word2vec Explained: Deriving Mikolov et al.'s Negative-Sampling Word-Embedding Method* (2014)
- Levy & Goldberg — *Neural Word Embedding as Implicit Matrix Factorization* (NeurIPS 2014)
- Gutmann & Hyvärinen — *Noise-contrastive estimation* (AISTATS 2010)
- Bengio & Senécal — *Adaptive Importance Sampling to Accelerate Training of a Neural Probabilistic Language Model* (IEEE TNN 2008)
- Rendle, Freudenthaler, Gantner, Schmidt-Thieme — *BPR: Bayesian Personalized Ranking from Implicit Feedback* (UAI 2009)
- Yi et al. — *Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations* (RecSys 2019)
- Karpukhin et al. — *Dense Passage Retrieval for Open-Domain Question Answering* (EMNLP 2020)
- Zhang, Chen, Wang, Yu — *Optimizing Top-N Collaborative Filtering via Dynamic Negative Item Sampling* (SIGIR 2013)
- Shi, Chen, Feng, Zhang, Wu, Gao, He — *On the Theories Behind Hard Negative Sampling for Recommendation* (WWW 2023)
- Fan, Bai, Pan, Hu, Zheng — *Batch-Mix Negative Sampling for Learning Recommendation Retrievers* (CIKM 2023)
- Chen, Kornblith, Norouzi, Hinton — *A Simple Framework for Contrastive Learning of Visual Representations* (ICML 2020)
- He, Fan, Wu, Xie, Girshick — *Momentum Contrast for Unsupervised Visual Representation Learning* (CVPR 2020)
- Wang, Zhang, Huang, Scott — *Cross-Batch Memory for Embedding Learning* (CVPR 2020)
- Prakash, Bermperidis, Chennu — *Evaluating Performance and Bias of Negative Sampling in Large-Scale Sequential Recommendation Models* (2024)
- Ma et al. — *Negative Sampling in Recommendation: A Survey and Future Directions* (2024)

Раздел 2. Глубокие архитектуры рекомендательных систем, адаптация и мультимодальные модели

Attention, архитектуры рекомендательных систем, параметро-эффективное обучение.

2.1 Двухбашенная модель (Two-tower)

Двухбашенная архитектура (Two-tower) — это подход на нейронных сетях, применяемый в рекомендательных системах для генерации кандидатов и retrieval (поиска кандидатов). Архитектура состоит из двух отдельных нейронных сетей («башен»), которые независимо кодируют пользователей и объекты в плотные эмбединги в общем векторном пространстве, где близость эмбедингов отражает релевантность.

Базовая идея — выучить отдельные представления для пользователей и объектов, которые затем можно эффективно сравнивать через векторную близость (обычно косинусное сходство или скалярное произведение). Такое разделение позволяет заранее вычислять эмбединги объектов и в реальном времени выполнять retrieval с помощью приближённого поиска ближайших соседей.

Архитектура и её варианты

Базовая двухбашенная архитектура

```

flowchart TB
    subgraph User_Tower [User Tower]
        UF[User Features] --> UE1[Dense Layer]
        UE1 --> UE2[Dense Layer]
        UE2 --> UEmb[User Embedding]
    end

    subgraph Item_Tower [Item Tower]
        IF[Item Features] --> IE1[Dense Layer]
        IE1 --> IE2[Dense Layer]
        IE2 --> IEmb[Item Embedding]
    end

    UEmb --> Sim[Similarity Score]
    IEmb --> Sim
    Sim --> Loss[Contrastive Loss]

```

Обе башни обычно содержат:

- **Входной слой:** конкатенация признаков или lookup эмбеддингов
- **Скрытые слои:** несколько полносвязных слоёв с функциями активации (ReLU, GELU)
- **Выходной слой:** плотный эмбеддинг фиксированной размерности (64–512)
- **Нормализация:** L2-нормализация для косинусного сходства

Варианты архитектуры

- Асимметричные башни: разные архитектуры для пользовательской и объектной башен — более простая для пользователей (фокус на поведении) и более сложная для объектов (обработка богатых признаков).
- Иерархические башни: многоуровневые эмбеддинги (категория, подкатегория, объект).
- Башни с attention: self-attention внутри башни для последовательных данных или cross-attention между историей пользователя и признаками объекта (используется редко).

Входные данные и их подготовка

Входы пользовательской башни

- Статические признаки: демографические и профильные данные.
- Поведенческие признаки: история взаимодействий, агрегированные статистики, временные паттерны.
- Последовательные признаки: недавняя история взаимодействий, сессионные взаимодействия, признаки на основе времени (свежесть, сезонность).
- Контекстные признаки: контекст текущей сессии, поисковые запросы, текущий контекст просмотра.

Входы объектной башни

- Контентные признаки: текстовые описания, заголовки, ключевые слова, эмбеддинги изображений.
- Метаданные, метрики популярности, метрики автора/источника.

Важно отметить, что в башнях не может быть interaction-признаков, так как башни должны быть независимы друг от друга.

Предобработка данных

- Feature engineering: извлечение текстовых и графических эмбеддингов, эмбеддинг-слои для признаков высокой кардинальности, нормализация непрерывных числовых признаков.
- Обработка последовательностей: агрегация истории пользователя (среднее, attention-взвешенный пулинг, RNN-кодирование), усечение/паддинг (последовательности фиксированной длины для батчевой обработки), временное кодирование (позиционные эмбеддинги, факторы временного затухания).

Обучение

Contrastive обучение

$$\mathcal{L} = -\log \frac{\exp(\mathbf{u} \cdot \mathbf{i}^+ / \tau)}{\exp(\mathbf{u} \cdot \mathbf{i}^+ / \tau) + \sum_j \exp(\mathbf{u} \cdot \mathbf{i}_j^- / \tau)}$$

Где:

- $\mathbf{u}$ — эмбеддинг пользователя
- $\mathbf{i}^+$ — эмбеддинг позитивного объекта
- $\mathbf{i}_j^-$ — эмбеддинги негативных объектов
- τ — температурный параметр

Можно использовать loss типа Hinge: $Loss = \max(0, margin - (u \cdot v_{positive}) + (u \cdot v_{negative}))$. Даёт качественные эмбеддинги, но требует значительных вычислительных ресурсов.

Binary Cross-Entropy Loss (pointwise)

Каждая пара (пользователь, объект) рассматривается как задача бинарной классификации (взаимодействовал/не взаимодействовал). Требуется явных позитивных и негативных меток. $Loss = -[y * \log(\sigma(u \cdot v)) + (1 - y) * \log(1 - \sigma(u \cdot v))]$, где $y=1$ для позитива, $y=0$ для негатива, σ — сигмоида.

In-batch Softmax Loss (sampled softmax / listwise)

Для каждого пользователя (с позитивным объектом) в батче все остальные объекты этого же батча (или отдельного батча негативов) считаются негативами. Вычисляется скалярное произведение эмбединга пользователя с эмбедингом позитивного объекта и со всеми эмбедингами негативов. К полученным оценкам применяется softmax. Loss — отрицательный логарифм вероятности позитивного объекта. $Loss = -\log(\exp(u \cdot v_{positive}/\tau) / (\exp(u \cdot v_{positive}/\tau) + \sum \exp(u \cdot v_{negative_j}/\tau)))$ Очень вычислительно эффективно, но зависит от негативов в батче и требует logQ-коррекции.

Стратегии сэмплирования

Позитивное сэмплирование:

- **Явный фидбэк:** прямые взаимодействия (клики, покупки, лайки)
- **Неявный фидбэк:** просмотры, проведённое время, доля досматривания
- **Временное взвешивание:** недавние взаимодействия имеют больший вес

Стратегии «negative sampling»:

1. **Случайное сэмплирование:** равномерное сэмплирование из всех объектов
2. **Сэмплирование по популярности:** популярные объекты сэмплируются чаще
3. **Hard Negative Mining:** сэмплирование объектов, похожих на позитивные, но без взаимодействия
4. **In-batch sampling:** в качестве негативов используются другие объекты из того же батча

In-batch negatives: для батча из N пар (пользователь, объект) позитивный объект каждого пользователя выступает негативом для остальных пользователей в батче. Это даёт $N \times (N-1)$ негативных пар на батч.

«logQ-коррекция»

Inference и serving

Двухфазный serving

Оффлайн-фаза:

1. **Вычисление эмбедингов объектов:** предварительный расчёт эмбедингов всех объектов.
2. **Построение индекса:** построение индекса приближённого поиска ближайших соседей (ANN — FAISS, Annoy, ScaNN).
3. **Кэширование:** хранение эмбедингов в быстрых системах retrieval.

Онлайн-фаза:

1. **Эмбединг пользователя:** вычисление пользовательского эмбединга через user-башню.
2. **Retrieval кандидатов:** запрос к ANN-индексу для выбора top-k похожих объектов.
3. **Постобработка:** применение бизнес-правил, фильтров разнообразия.

Практические соображения

- Использовать dropout и batch normalization для регуляризации.
- Экспериментировать с разными размерностями эмбедингов (64, 128, 256, 512).
- Применять смешанные стратегии negative sampling.
- Применять «logQ-коррекцию» при смещённом сэмплировании.
- Заранее вычислять и кэшировать эмбединги объектов.

2.2 Wide & Deep Learning

Wide & Deep Learning — это совместная архитектура, разработанная Google для рекомендательных систем и поискового ранжирования. Она объединяет линейную (wide) модель с глубокой нейронной сетью, чтобы получить одновременно запоминание и обобщение.

- **Запоминание (memorization):** обучение частым совместным появлениям объектов или признаков, чтобы фиксировать прямые, явные связи в исторических данных.
- **Обобщение (generalization):** перенос выученных паттернов на новые комбинации объектов и исследование новых комбинаций признаков.

Архитектура

- **Wide-компонент:** линейная модель, использующая признаки, one-hot закодированные категориальные признаки и комбинации признаков (конкатенация значений категориальных признаков).
- **Deep-компонент:** feed-forward нейронная сеть, принимающая на вход плотные эмбединги или числовые признаки. Обычно состоит из нескольких полносвязных слоёв с функциями активации (ReLU).
- **Совместное обучение:** выходы wide- и deep-компонентов комбинируются взвешенной суммой в итоговое предсказание.
- Обучается end-to-end с единой функцией потерь (logistic loss для классификации, регрессия для числовых предсказаний).

Преимущества

- Объединяет способность линейных моделей к запоминанию с обобщающими способностями глубоких сетей.
- Лучше работает одновременно с разреженными и плотными признаками.
- Масштабируется.
- Гибкий feature engineering.

Недостатки

- Для wide-компонента всё ещё требуется ручной feature engineering.
- Обучение вычислительно затратно, возможен оверфит.

Возможные улучшения

- DeepFM: комбинирует машины факторизации с глубокими сетями, заменяя wide-компонент.
- «Deep & Cross Network»: использует явные слои кросс-признаков вместо ручного feature engineering.

Ссылки

- [Original Paper](#)
- [TensorFlow Implementation](#)
- [Google AI Blog Post](#)

2.3 Deep & Cross Network

Deep & Cross Network (DCN) — нейросетевая архитектура от Google, объединяющая cross-сеть с глубокой сетью, чтобы эффективно выучивать явные и неявные взаимодействия признаков. Оригинальный DCN был представлен в 2017 году, улучшенная версия DCN v2 — в 2020 году.

Архитектура

- Cross-сеть: принимает входной вектор эмбеддингов и вычисляет кросс-произведения с самим собой, каждый слой увеличивает полиномиальную степень на единицу. Размерность входа сохраняется по всей сети.
- Deep-сеть: MLP с функцией активации ReLU.
- Комбинация: выходы конкатенируются и пропускаются через финальный плотный слой, дающий предсказание (обычно вероятность).

Улучшения DCN v2

- Заменяет векторную cross-операцию на матричную с использованием низкоранговой декомпозиции.
- Использует Mixture of Experts для разложения выученной матрицы на подпространства, которые затем агрегируются через гейты.

==

Преимущества

- Автоматически выучивает взаимодействия признаков без ручного feature engineering.
- Меньшая вычислительная сложность по сравнению с сопоставимыми моделями.

Недостатки

- Оригинальная cross-сеть ограничена в типах взаимодействий, которые может выучить.
- Подбор гиперпараметров может быть непростым.

Ссылки

- [Original DCN Paper \(2017\)](#)
- [DCN v2 Paper \(2020\)](#)
- [TensorFlow Implementation](#)

2.4 DLRM (Deep Learning Recommendation Model)

Deep Learning Recommendation Model — нейросетевая архитектура, разработанная Facebook для эффективной работы с крупномасштабными рекомендательными задачами. Она ориентирована на объединение числовых признаков и категориальных эмбеддингов вычислительно оптимизированным образом. DLRM явно моделирует попарные взаимодействия признаков через скалярные произведения, формируя как взаимодействия первого порядка, так и более высоких порядков.

Архитектура

- Эмбеддинг-слои (для разреженных признаков): категориальные признаки отображаются в эмбеддинги одинаковой размерности.
- Bottom MLP (для плотных признаков): пропускает числовые признаки через MLP, на выходе — представления той же размерности, что и эмбеддинги.

- Взаимодействие признаков: берутся все пары эмбединговых векторов и вычисляются скалярные произведения между ними, после чего все они конкатенируются.
- Top MLP: выход предыдущего шага пропускается через MLP и формируется итоговое предсказание с функцией активации sigmoid.

```

flowchart TD
    subgraph Input
        SF[Sparse Features]
        DF[Dense Features]
    end

    subgraph Processing
        SF --> EL[Embedding Layers]
        DF --> BMLP[Bottom MLP]
    end

    subgraph Interaction
        EL --> FI[Feature Interactions<br>Dot Products]
        BMLP --> CC[Concatenate with<br>Interactions]
        FI --> CC
        EL --> CC
    end

    CC --> TMLP[Top MLP]
    TMLP --> Out[Prediction]

```

Преимущества

- Эффективно работает одновременно с разреженными и плотными признаками и, в отличие от чисто глубоких моделей, явно моделирует взаимодействия признаков.
- Хорошо масштабируется для продакшен-рекомендательных систем.

Недостатки

- Может иметь большой расход памяти из-за таблиц эмбедингов.
- Может не полностью улавливать сложные нелинейные связи между признаками.

Ссылки

- [Original Paper](#)
- [Official PyTorch Implementation](#)

2.5 Neural Collaborative Filtering (NCF)

Neural Collaborative Filtering (NCF) — это подход на основе глубокого обучения к коллаборативной фильтрации в рекомендательных системах, цель которого — выучить сложную функцию взаимодействия пользователя и объекта с помощью нейронных сетей. NCF преодолевает ограничения классической «матричной факторизации», используя нелинейные нейронные сети для моделирования взаимодействий пользователя и объекта.

Архитектура

- Вход: one-hot ID пользователя и ID объекта.
- Эмбединг-слой преобразует разреженные one-hot векторы в плотные эмбединги пользователей и объектов.
- Generalized Matrix Factorization (GMF): поэлементное произведение эмбедингов пользователя и объекта с последующей сигмоидой.
- Multi-Layer Perceptron (MLP): конкатенирует эмбединги пользователя и объекта, пропускает их через несколько скрытых слоёв с функцией активации ReLU и финальную сигмоиду.
- Neural Matrix Factorization (NeuMF): конкатенирует выходы GMF и MLP, применяет проекционный слой и сигмоиду.

Подход использует binary cross-entropy loss для неявного фидбэка (факт взаимодействия пользователя с объектом). «Negative sampling»: равномерное сэмплирование или стратегии на основе популярности.

Преимущества

- Улавливает сложные нелинейные взаимодействия между пользователями и объектами.
- Более выразителен, чем классическая «матричная факторизация».
- Гибкая архитектура, допускающая различные нейросетевые дизайны.

Недостатки

- Требуется больше обучающих данных и вычислительно дороже классических методов.
- Может перефититься на разреженных датасетах.

Возможные улучшения

- Предобучить компоненты GMF и MLP по отдельности и использовать предобученные веса для инициализации NeuMF.
- **Neural Graph Collaborative Filtering (NGCF)**: добавляет GNN и моделирует связи высокого порядка в графах взаимодействий пользователь—объект.

Ссылки

- Paper
- Official implementation
- PyTorch implementation

2.6 logQ-коррекция

logQ-коррекция — техника коррекции смещения, применяемая в рекомендательных системах для учёта неравномерного сэмплирования во время обучения. Когда негативы выбираются не равномерно случайно (например, при сэмплировании по популярности или in-batch sampling), модель учит смещённое представление, в котором преобладают часто сэмплируемые объекты.

logQ-коррекция модифицирует обучающий критерий, вычитая логарифм вероятности сэмплирования каждого объекта как негатива:

Исходный loss (contrastive):

$$\mathcal{L} = -\log \frac{\exp(s_{u,i^+})}{\exp(s_{u,i^+}) + \sum_{j \in \text{negatives}} \exp(s_{u,j})}$$

Скорректированный loss: $\mathcal{L}_{\text{corrected}} = -\log \frac{\exp(s_{u,i^+})}{\exp(s_{u,i^+}) + \sum_{j \in \text{negatives}} \exp(s_{u,j}) - \log Q(j)}$

Где:

- $s_{u,i}$ — оценка близости между пользователем u и объектом i
- $Q(j)$ — вероятность сэмплирования объекта j в качестве негатива

logQ-коррекция фактически «дисконтирует» оценки близости часто сэмплируемых объектов. Если объект сэмплируется с высокой вероятностью $Q(j)$, то $\log Q(j)$ менее отрицателен, поэтому $s_{u,j} - \log Q(j)$ становится меньше, и влияние этого объекта в loss уменьшается.

При равномерном сэмплировании: $Q(j) = \frac{1}{N}$ для всех объектов, поэтому $\log Q(j)$ — константа и сокращается.

При сэмплировании по популярности: $Q(j) \propto \text{popularity}(j)$, поэтому для популярных объектов поправки больше.

Вычисление Q(j)

In-batch sampling: если в качестве негативов используются другие объекты в батче:

$$Q(j) = \frac{\text{frequency of item } j \text{ in training data}}{\text{total interactions}}$$

Сэмплирование по популярности: если негативы сэмплируются пропорционально популярности: $Q(j) = \frac{\text{interaction_count}(j)}{\sum_k \text{interaction_count}(k)}$

Ссылки

- Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations
- On Sampling Strategies for Neural Network-based Collaborative Filtering

2.7 LoRA (Low-Rank Adaptation)

LoRA (Low-Rank Adaptation) — параметро-эффективная техника дообучения, позволяющая адаптировать крупные предобученные модели к конкретным задачам при минимальных вычислительных затратах.

Основная идея

У предобученных моделей весовые матрицы большие, однако ранг матриц обновлений (изменений) при дообучении может быть существенно ниже. LoRA делает следующее:

1. Замораживает исходные веса предобученной модели.
2. Внедряет в слои модели обучаемые матрицы низкоранговой декомпозиции.
3. Представляет обновления весов как произведение этих меньших матриц.

Математически, вместо прямого обучения матрицы обновления ΔW , которая прибавляется к предобученной матрице весов W , LoRA аппроксимирует это обновление двумя меньшими матрицами: $\Delta W = BA$.

Где:

- A — матрица размера $r \times d$ ($r \ll d$)
- B — матрица размера $d \times r$ ($r \ll d$)
- r — параметр ранга (обычно 4–256)

Во время инференса эффективная матрица весов: $W' = W + \Delta W = W + BA$.

LoRA обычно применяется к избранным весовым матрицам внутри transformer-архитектур:

- Чаще всего — к attention-слоям (проекции query, key, value и выходной).
- Может применяться также к feed-forward слоям.
- Обычно A инициализируется случайной гауссовой инициализацией, а B — нулями.

Дополнительный масштабирующий параметр α управляет величиной обновлений: $W' = W + \alpha(BA)/r$.

Преимущества

1. Параметро-эффективность: уменьшает число обучаемых параметров до 10 000 раз по сравнению с полным дообучением для LLM.
2. Эффективность по памяти: уменьшает требования к GPU-памяти примерно в 3 раза по сравнению с полным дообучением.
3. Без задержек на инференсе: LoRA-матрицы после обучения можно слить с исходными весами, что не добавляет латентности на инференсе.
4. Сопоставимое или лучшее качество: часто работает наравне с полным дообучением или лучше него, несмотря на меньшее число обучаемых параметров.
5. Переключение задач: упрощает переключение между задачами за счёт замены только LoRA-весов вместо полных чекпойнтов модели.

Ключевые гиперпараметры

1. Ранг (r): определяет ёмкость адаптации. Более высокие ранги дают больше гибкости, но увеличивают число параметров. Обычные значения — 4–256, но после 64 или 128 наступают убывающие отдачи. Для меньших моделей r может быть ниже, для больших — выше.
2. Alpha (α): масштабирующий коэффициент для LoRA-обновлений. Хорошая эвристика — установить alpha равным удвоенному рангу.
3. Целевые модули: к каким слоям модели применять LoRA. Применение LoRA к большему числу слоёв, как правило, улучшает качество, но увеличивает потребление памяти.
4. Dropout: можно применять к LoRA-матрицам для предотвращения оверфита.

Варианты

1. QLoRA: квантует базовую модель до 4-битной точности, чтобы дополнительно сократить требования к памяти.
2. Layer-wise Optimal Rank Adaptation: назначает разным слоям разные ранги в зависимости от их важности.
3. AdaLoRA: адаптивно распределяет бюджет параметров по слоям.
4. VeRA: Vector-based Random Matrix Adaptation, где низкоранговая декомпозиция использует случайные матрицы.

Ссылки

- [Original Paper: LoRA: Low-Rank Adaptation of Large Language Models](#)
- [QLoRA Paper: QLoRA: Efficient Finetuning of Quantized LLMs](#)
- [AdaLoRA Paper: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning](#)

2.8 CLIP (Contrastive Language-Image Pre-training)

CLIP (Contrastive Language-Image Pre-training) — нейросетевая модель, разработанная OpenAI, эффективно выучивающая визуальные понятия по обучающему сигналу на естественном языке. Это dual-encoder модель, которая совместно обучает энкодер изображений и энкодер текста так, чтобы для соответствующих пар получались близкие эмбединги. Модель обладает хорошими zero-shot возможностями, что позволяет применять её ко многим задачам без явного обучения под них.

Архитектура

- Энкодер изображений: Vision Transformer (ViT) или ResNet принимает изображения и выдаёт эмбединги фиксированной размерности (512 или 768).
- Энкодер текста: Transformer обрабатывает текст и выдаёт текстовые эмбединги фиксированной размерности. Максимальная длина контекста — 77 токенов.
- Оба энкодера проецируют свои выходы в общее эмбединг-пространство, эмбединги L2-нормализуются к единичной длине, близость вычисляется через скалярное произведение (для нормализованных векторов эквивалентно косинусному сходству).

Обучение

CLIP использует contrastive обучение на крупномасштабных парах изображение—текст:

- Батч содержит N пар изображение—текст.
- И изображения, и тексты кодируются в общее эмбединг-пространство.
- Вычисляется матрица близостей по всем N×N возможным комбинациям изображение—текст.
- С помощью contrastive loss максимизируется близость для корректных пар (на диагонали) и минимизируется для некорректных (вне диагонали).

Zero-shot классификация

- Для заданной задачи классификации изображений составляются описательные текстовые промпты для каждого класса.
- Входное изображение пропускается через энкодер изображений CLIP, чтобы получить его эмбединг.
- Каждый промпт класса пропускается через энкодер текста CLIP, чтобы получить текстовый эмбединг для каждого класса.
- Вычисляется косинусное сходство между эмбедингом изображения и эмбедингом каждого промпта класса.
- В качестве предсказания выбирается класс, промпт которого имеет максимальную близость с эмбедингом изображения.

Дообучение CLIP

- Полное дообучение — разморозить и обновлять все параметры обоих энкодеров. Требуется значительных вычислительных ресурсов, но даёт лучшее качество.
- Linear Probing — заморозить оба энкодера и обучать только линейный классификатор сверху. Работает неплохо, но хуже полного дообучения.
- Параметро-эффективное дообучение: «LoRA», адаптер-слои, prompt/prefix tuning.
- Few-shot обучение с задаче-специфичными промптами.

Практические соображения

- Обработка текста длиннее 77 токенов: усечение, разбиение на чанки и усреднение/пулинг эмбедингов, иерархическое кодирование (использование модели для обработки последовательности эмбедингов), скользящее окно с attention, сжатие текста.

Ссылки

- [Original CLIP Paper](#)
- [OpenAI CLIP GitHub Repository](#)
- [Hugging Face CLIP Documentation](#)

Раздел 3. Обработка естественного языка: базовые концепции и эмбединги

3.1 Введение в NLP (обзор задач и пайплайна)

Обработка естественного языка (Natural Language Processing, NLP) — это междисциплинарная область, объединяющая лингвистику, информатику и искусственный интеллект, цель которой — научить компьютеры понимать, интерпретировать и порождать человеческий язык.

До 2010-х большинство подходов опирались на признаки, сконструированные вручную (частоты слов, n-граммы, лингвистические разметки), извлекаемые из текста. На ранних этапах NLP основные подходы к решению задач сводились к if-else правилам и шаблонам, к грамматикам на правилах, онтологиям и эвристикам. Несколько примеров: - проверка того, присутствует ли слово (или его основа/лемма) в словаре; - проверка того, удовлетворяет ли фрагмент текста заранее заданному грамматическому шаблону; - применение Hidden Markov Models к разметке частей речи и Conditional Random Fields к распознаванию именованных сущностей (NER); - применение Naïve Bayes и SVM к подсчётам слов (включая TF-IDF) и n-граммам.

В 2010-х были введены эмбединги слов — Word2Vec, GloVe, fastText и рекуррентные модели стали использоваться шире. RNN и LSTM были предложены раньше, но именно вместе с эмбедингами слов их начали применять массово. Позднее популярность набрала разновидность GRU.

В 2017 году вышла статья [Attention is all you need](#), с которой началась эпоха Transformer. Трансформеры в стиле BERT и GPT стали ядром последующих подходов.

В 2018 году вышла ещё одна поворотная статья — [Universal Language Model Fine-tuning for Text Classification \(ULMFiT\)](#), которая ввела дообучение (fine-tuning) в современном виде: общее предобучение, дообучение под целевую задачу и дообучение классификатора целевой задачи.

Похожие идеи исследовались и в других работах. В [Large Language Models in Machine Translation](#) авторы обучали модель на корпусе до 2 трлн токенов с до 300 млрд n-грамм (вплоть до 5-грамм). А [Semi-supervised Sequence Learning](#) предлагала предобучить sequence-автоэнкодер и затем дообучить его под классификацию.

Появление больших языковых моделей (Large Language Models, LLM) стало ещё одним значимым шагом. Критерии LLM размыты, поэтому даже BERT можно называть LLM, но если под LLM понимать крупные модели, способные решать широкий спектр задач через промптинг и без дообучения (или с минимальным дообучением), то первыми LLM можно считать GPT-2 или GPT-3.

Оставшаяся часть этой заметки служит индексом по другим заметкам, связанным с NLP.

Обработка текста

Сырые тексты необходимо обрабатывать, прежде чем подавать их в ML-модели. Эта обработка включает: - Стемминг и лемматизацию — приведение слов к базовым формам и поиск корней слов соответственно; - Удаление стоп-слов — избавление от слов, несущих мало информации; - Токенизацию — разбиение текста на элементы (символы, фрагменты слов, слова) и их кодирование.

Архитектуры - Рекуррентные нейросети (RNN), LSTM, GRU - Transformer - T5

Техники обучения и дообучения - Цели предобучения - Стратегии дообучения - Instruction Learning - Reinforcement Learning from Human Feedback (RLHF) - Direct Preference Optimization (DPO) - Токенизация - Parameter Efficient Fine-tuning (LoRA, QLoRA, Prefix Tuning) - Промптинг

Prompt Tuning: настраивает набор конкатенируемых входных эмбедингов (обычно их называют «soft prompts», но речь не о soft prompts в широком смысле). Изначально применялся к моделям T5-LM.

Prefix Tuning: настраивает KV-кэш (soft prefixes) на каждом слое; неформально его можно описать как «prompt tuning, но на каждом слое», хотя это и не вполне точно. На практике используется вспомогательный MLP для генерации soft prefixes — это облегчает обучение. Изначально применялся к моделям GPT-2 и BART.

P-Tuning: использует LSTM для генерации soft prompts (не prefixes). Изначально применялся к GPT-2 и моделям BERT/RoBERTa/MegatronLM.

P-Tuning v2: по сути Prefix Tuning, применённый к моделям типа BERT.

Задачи NLP

- Распознавание именованных сущностей (NER)
- Машинный перевод
- Классификация текстов
 - Topic Modeling — выделение абстрактных тем в коллекции документов без учителя
 - Методы Topic Modeling — каталог методов с фреймворком выбора
 - LDA — классическая вероятностная генеративная модель
 - BERTopic — современный пайплайн на эмбедингах
- Генерация текста
- Чат-боты
- RAG
- Поиск и извлечение информации
 - Поиск
 - Вопросно-ответные системы
 - Суммаризация

Метрики - BLEU - ROUGE - METEOR - Perplexity - Методы оценки человеком - Бенчмарк-дасеты

Ссылки - NLP for Supervised Learning - A Brief Survey

3.2 Эмбединги слов (общая идея)

Эмбединг слова — это представление слова, как правило, в виде вектора значений.

Простейший способ представления слов — one-hot кодирование: i -е слово словаря получает вектор с единицей в i -й позиции и нулями во всех остальных. Но это неэффективно: такие векторы слишком разреженные, слишком длинные и несут слишком мало информации.

Чуть лучше работают подходы на основе подсчётов и TF-IDF.

Первой работой, предложившей использовать эмбединги, была [A Neural Probabilistic Language Model](#) (2003): авторы обучали базовую языковую модель, чтобы получить распределённое представление слов. Другая ранняя работа — [A unified architecture for natural language processing: deep neural networks with multitask learning](#) (2008), где слова «вкладывались в d -мерное пространство через lookup-таблицу» — фактически первое современное упоминание эмбедингов слов.

Но именно статья по [Word2Vec Efficient Estimation of Word Representations in Vector Space](#) стала началом массового использования эмбедингов слов. Word2Vec давал эффективный способ обучать неглубокие нейросети на больших корпусах и получать многомерные векторы такие, что слова, встречающиеся в схожих контекстах, имеют схожие векторы.

GloVe — [GloVe: Global Vectors for Word Representation](#) — использовал глобальную статистику совместной встречаемости слов для построения эмбедингов, применяя иной подход (матричную факторизацию и статистику по всему корпусу).

fastText — [Enriching Word Vectors with Subword Information](#) — расширил Word2Vec, представляя каждое слово как мешок символьных n -грамм; это позволило модели порождать векторы для редких или ошибочно написанных слов, складывая subword-информацию.

Позднее для эмбедингов слов начали использовать модели типа BERT. ELMo выдаёт контекстные эмбединги.

3.3 Word2Vec

Word2Vec — это подход к обучению эмбедингов слов. Он стал первым массово популярным способом представления слов в виде плотных векторов в непрерывном векторном пространстве, где семантически близкие слова располагаются ближе друг к другу.

Word2Vec использует неглубокие нейросети с одним скрытым слоем для обучения векторных представлений: либо предсказывая слово по его контексту (Continuous Bag-of-Words), либо предсказывая контекстные слова по целевому слову (Skip-gram).

Модель Skip-gram принимает на вход центральное слово и предсказывает окружающие контекстные слова. Этот вариант лучше работает для редких слов и более крупных датасетов. Модель CBOW принимает на вход контекстные слова и предсказывает центральное слово. Это даёт более быстрое обучение и лучше работает для частотных слов.

Архитектура

1. Преобразовать входное слово в one-hot вектор.
2. Умножить на матрицу весов W размера $V \times D$ (что эквивалентно lookup).
3. Для CBOW: усреднить полученные эмбединги.
4. Умножить на контекстную матрицу W' размера $D \times V$.
5. Применить softmax, чтобы получить распределение вероятностей.

Оптимизации обучения

- Заменить полный softmax иерархическим softmax.
- Не обновлять все веса на каждой итерации, а использовать negative sampling — обновлять только положительные и отрицательные примеры.
- Случайно отбрасывать частотные слова во время обучения.

[!example]- Реализация на PyTorch

```
python class Word2Vec(nn.Module):
    def __init__(self, vocab_size, embedding_dim, model_type):
        super(Word2Vec, self).__init__()
        self.embeddings = nn.Embedding(vocab_size, embedding_dim)
        self.linear = nn.Linear(embedding_dim, vocab_size)
        self.model_type = model_type
```

```
def forward(self, context_words):
    # context_words: (batch_size, context_size)
    embeds = self.embeddings(context_words)
    if self.model_type == 'CBOW':
        embeds = embeds.mean(dim=1)
    out = self.linear(embeds)
    return out
```

Ссылки

- [Original Paper: Efficient Estimation of Word Representations in Vector Space](#)
- [Second Paper: Distributed Representations of Words and Phrases and their Compositionality](#)
- [Gensim Word2Vec Documentation](#)

3.4 GloVe

GloVe (Global Vectors for Word Representation) — техника эмбедингов слов, разработанная в 2014 году. Если Word2Vec изучает совместную встречаемость слов через скользящее окно (локальная статистика), то GloVe — через матрицу совместной встречаемости (глобальная статистика). Затем GloVe обучает векторы слов так, чтобы их разности предсказывали отношения совместной встречаемости. Несмотря на различные исходные точки, представления слов у Word2Vec и GloVe оказываются похожими.

Первый шаг — построить матрицу совместной встречаемости слов «word-word» на основе корпуса. Обучение проводится по ненулевым совместным встречаемостям. Целевая функция оптимизирует векторы слов так, чтобы их скалярное произведение было равно логарифму вероятности совместной встречаемости этих слов.

Ссылки

- [Original Paper: GloVe: Global Vectors for Word Representation](#)
- [Stanford NLP GloVe Project Page](#)
- [NLP Course | For You: GloVe](#)

3.5 fastText

fastText — это библиотека от Facebook для эффективной классификации текстов и обучения векторных представлений слов. Она расширяет Word2Vec, включая subword-информацию, что позволяет получать осмысленные эмбединги для редких и OOV-слов (out-of-vocabulary).

Например, слово «apple» может быть представлено как: само слово `apple` и символьные n-граммы `ap`, `app`, `appl`, `apple`, `ppl`, `pple`, `ple`, `le`. У каждой из этих n-грамм своё векторное представление, а итоговый вектор слова — это сумма subword-векторов и вектора самого слова.

Ссылки

- [Official fastText Website](#)
- [Original Paper: Enriching Word Vectors with Subword Information](#)
- [Text Classification Paper: Bag of Tricks for Efficient Text Classification](#)

3.6 TF-IDF (Term Frequency-Inverse Document Frequency)

Term Frequency-Inverse Document Frequency (TF-IDF) — численная статистика, используемая в NLP для оценки того, насколько важно слово (терм) для документа в корпусе. Эта величина растёт пропорционально числу появлений слова в документе и компенсируется частотой слова во всём корпусе.

Term Frequency (TF)

Измеряет, насколько часто терм встречается в конкретном документе. Предполагается, что слова, встречающиеся в документе чаще, важнее для этого документа. Есть несколько способов её посчитать:

- **Сырой подсчёт (Raw Count):** простейший вариант. $TF(t, d) = f(t, d)$, где $f(t, d)$ — сырой подсчёт термина в документе.
- **Булева частота (Boolean Frequency):** $TF(t, d) = 1$, если t присутствует в d , иначе 0.
- **Логарифмическое масштабирование:** $TF(t, d) = \log(1 + f_{t,d})$.
- **Аугментированная частота (Augmented Frequency):** нормирует сырую частоту по частоте самого частого термина в документе, чтобы не было смещения в пользу более длинных документов: $TF(t, d) = 0.5 + 0.5 \cdot \frac{f_{t,d}}{\max_{t' \in d} f_{t',d}}$

Inverse Document Frequency (IDF)

Измеряет, сколько информации несёт терм по всему корпусу. Оценивает «информативность» или «редкость» терма.

Стандартная формула: $IDF(t, D) = \log(N/df(t))$

Где: * N : общее число документов в корпусе D . $N = |D|$. * $df(t)$: Document Frequency терма — число документов корпуса, содержащих этот терм.

Сглаживание: чтобы избежать деления на ноль (если $df(t) = 0$) и чтобы IDF не обнулялся для термов, встречающихся во всех документах ($df(t) = N$), обычно применяют сглаживание. Распространённый вариант: $IDF(t, D) = \log(N/(df(t) + 1)) + 1$ Scikit-learn по умолчанию добавляет 1 к числителю: $IDF(t, D) = \log((N + 1)/(df(t) + 1)) + 1$

Вычисление TF-IDF

$$TF - IDF(t, d, D) = TF(t, d) * IDF(t, D)$$

Векторизация через TF-IDF

TF-IDF используется для преобразования коллекции текстовых документов в численную матрицу признаков (матрицу «документ–терм»). Шаги следующие: 1. **Создание словаря:** найти все уникальные термы по всему корпусу. 2. **Построение матрицы:** создать матрицу, где строки соответствуют документам, а столбцы — термам из словаря. 3. **Заполнение матрицы:** значение в ячейке (i, j) — это TF-IDF-оценка j -го терма в i -м документе. 4. **Результат:** получается разреженная матрица: большинство значений равны нулю, поскольку отдельный документ обычно содержит лишь небольшое подмножество общего словаря.

Ссылки

- [Wikipedia: tf-idf](#)
- [Scikit-learn: TfidfVectorizer Documentation](#)

3.7 Рекуррентные нейросети (RNN)

Рекуррентные нейронные сети (RNN) — тип нейросетей, спроектированных для обработки последовательных данных. В отличие от feedforward-сетей, у RNN связи образуют направленные циклы, что позволяет им хранить внутреннее состояние, удерживающее информацию о предыдущих входах в последовательности.

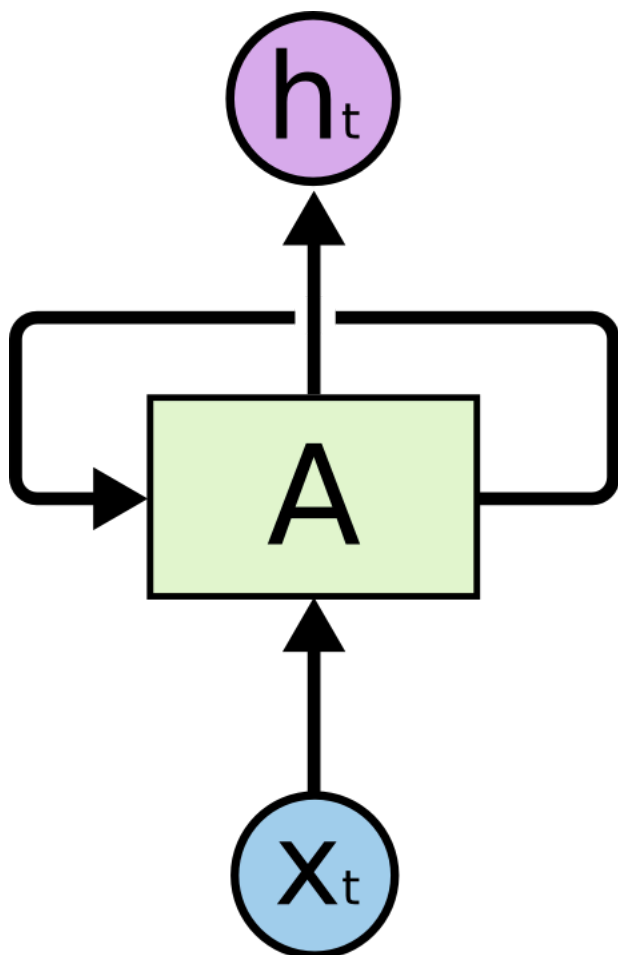
Основная идея RNN — обрабатывать последовательные данные по одному элементу за раз, сохраняя информацию о предыдущих элементах в скрытом состоянии.

```
graph LR
    A[Input x<sub>t</sub>] --> B((RNN Cell))
    B --> C[Hidden State h<sub>t-1</sub>]
    C --> B
    B --> D[Output y<sub>t</sub>]
    B --> E[Hidden State h<sub>t</sub>]
    E --> F[Next Timestep]
```

Архитектура RNN

На каждом шаге времени t базовая RNN-ячейка: 1. Принимает текущий вход x_t и предыдущее скрытое состояние h_{t-1} . 2. Вычисляет новое скрытое состояние h_t . 3. Выдаёт выход y_t .

Математическая запись: $h_t = \tanh(W_{xh} x_t + b_{xh} + W_{hh} h_{t-1} + b_{hh})$ $y_t = \text{softmax}(W_{hy} h_t + b_{hy})$



=

Варианты RNN

- Двухнаправленная RNN (Bidirectional RNN) обрабатывает последовательность в прямом и обратном направлениях, чтобы захватить контекст и из прошлого, и из будущего.
- Глубокая (стэкированная) RNN складывает несколько слоёв RNN друг над другом: выходная последовательность одного слоя становится входной последовательностью для следующего.

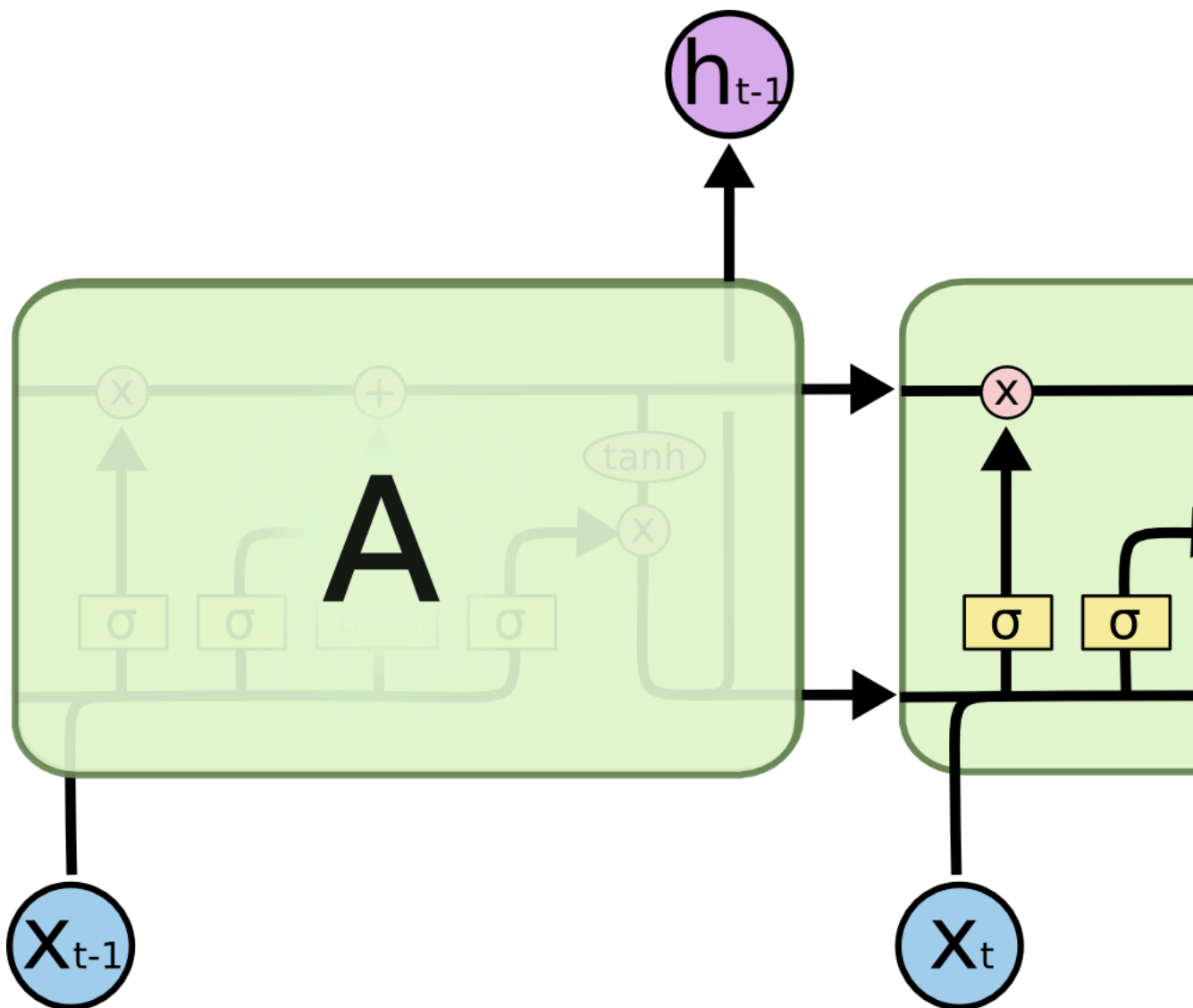
Типичные проблемы

- Стандартные RNN страдают от затухающего или взрывного градиента при backpropagation through time, из-за чего их трудно обучать на длинных последовательностях. Возможные решения: gradient clipping, лучшая инициализация весов, специализированные архитектуры — LSTM и GRU.
- Стандартные RNN плохо схватывают долгие зависимости в последовательностях.

Архитектура LSTM

LSTM решает проблему затухающего градиента за счёт введения cell state и трёх гейтов:

Input Gate: управляет тем, какую новую информацию сохранять. Forget Gate: управляет тем, какую информацию отбрасывать. Output Gate: управляет тем, какую информацию выдавать на выход.



Математическая запись:

$$f_t = \sigma(W_{xf} x_t + b_{xf} + W_{hf} h_{t-1} + b_{hf}) \quad i_t = \sigma(W_{xi} x_t + b_{xi} + W_{hi} h_{t-1} + b_{hi}) \quad \tilde{C}t = \tanh(W_{xc} x_t + b_{xc} + W_{hc} h_{t-1} + b_{hc}) \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}t \quad o_t = \sigma(W_{xo} x_t + b_{xo} + W_{ho} h_{t-1} + b_{ho}) \quad h_t = o_t \odot \tanh(C_t)$$

Архитектура GRU

GRU — упрощённый вариант LSTM с меньшим числом параметров. Update Gate: объединяет input и forget гейты. Reset Gate: управляет тем, какую долю предыдущего состояния забыть.



$$\text{Математическая запись: } z_t = \sigma(W_{xz} x_t + b_{xz} + W_{hz} h_{t-1} + b_{hz}) \quad r_t = \sigma(W_{xr} x_t + b_{xr} + W_{hr} h_{t-1} + b_{hr}) \quad \tilde{h}_t = \tanh(W_{xh} x_t + b_{xh} + r_t \odot (W_{hh} h_{t-1} + b_{hh})) \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Ссылки - Colah's Blog: Understanding LSTM Networks - The Unreasonable Effectiveness of Recurrent Neural Networks - LSTM Original Paper - GRU Paper - PyTorch RNN Documentation

Раздел 4. Трансформер, BERT и Retrieval-Augmented Generation

4.1 Архитектура Transformer

Первый Transformer был представлен в статье «Attention Is All You Need», вскоре после чего был опубликован BERT.

В этом разделе рассматриваются различные способы построения трансформеров и их эволюция со временем.

Vanilla Transformer

Энкодер состоит из 6 одинаковых слоёв, в каждом из которых два подслоя: multi-head self-attention (само-внимание с несколькими головами) и поэлементная (position-wise) полносвязная нейронная сеть прямого распространения. После каждого подслоя применяются остаточное соединение и layer normalization. Все выходы имеют размерность 512.

Декодер также содержит 6 аналогичных слоёв, но в нём добавлен третий подслой — multi-head attention над выходом энкодера. В нём используется тот же подход с остаточными соединениями и нормализацией. Кроме того, в self-attention декодера маскируются будущие позиции, чтобы предсказания для следующей позиции зависели только от предшествующих позиций.

Вариации

Токенизаторы

Токенизатор	Описание
BPE	Byte-Pair Encoding. Итеративно объединяет наиболее частые пары символов или подслов; слова разбиваются на максимально длинные известные подсловные фрагменты. <i>Примеры:</i> GPT-2, RoBERTa.
WordPiece	Оптимизированный BPE — сопоставляет максимально длинные подстроки с начала. <i>Примеры:</i> BERT.
SentencePiece	<i>Примеры:</i> T5, ALBERT.

Позиционные эмбединги

Позиционные эмбединги добавляются к токен-эмбедингам. Опционально могут добавляться и сегментные эмбединги — в BERT сегментные эмбединги разделяют пары предложений.

Тип	Описание
Sinusoidal	Фиксированные. Расширяются для более длинных последовательностей, но обычно работают хуже обучаемых эмбедингов.
Learned absolute	Вектор на каждую позицию; максимальная длина фиксируется во время обучения. Добавляются внутри механизма внимания.
Relative	Кодируют расстояние между токенами (например, B на N шагов впереди A).
T5 bias	Обучает скалярное смещение, добавляемое непосредственно к логитам внимания на основе относительного расстояния между запросом (query) и ключом (key).
RoPE	Rotary Positional Embeddings. Позиционно-зависимые вращения, применяемые к фрагментам Q/K-векторов перед скалярным произведением.

Стратегии декодирования

Стратегия	Описание
Greedy search	На каждом шаге выбирается единственный токен с наибольшей вероятностью. Простой и детерминированный подход; склонен к повторам и невыразительному выводу. Локально оптимальные выборы могут приводить к плохой последовательности в целом.
Beam search	Поддерживается k параллельных кандидатов-последовательностей («лучи»); на каждом шаге каждый луч расширяется, после чего сохраняются топ- k по кумулятивной логарифмической вероятности. Даёт более связный текст, чем greedy, но всё ещё может повторяться.
Temperature sampling	Логиты пересчитываются с делением на T перед softmax — $\text{softmax}(\text{logits}/T)$. $T > 1$ увеличивает случайность, $T < 1$ уменьшает её; $T \rightarrow 0$ приближает к greedy. Более высокая T полезна для творческой генерации.
Top-K sampling	На каждом шаге рассматриваются только K наиболее вероятных токенов. Дешёвый способ отсечь нереалистичный хвост распределения.
Top-P (nucleus) sampling	Выбирается наименьшее множество токенов, кумулятивная вероятность которого превышает P . Более адаптивный подход, чем Top-K, для контекстов с разной энтропией.

Известные архитектуры

Модель	Описание
ALBERT	A Lite BERT. Факторизованная параметризация эмбедингов (матрица эмбедингов словаря разбивается на две меньшие), межслойное разделение параметров, функция потерь на согласованность между предложениями.
Alpaca	LLaMA, дообученная на ~52 тыс. синтетических примеров следования инструкциям.
BART	Bidirectional and Auto-Regressive Transformers. Двухнаправленный энкодер BERT в паре с авторегрессионным декодером GPT.
CLIP	Contrastive Language-Image Pre-training. Двухэнкодерная архитектура (энкодер изображений ViT или ResNet + текстовый Transformer-энкодер), обученная контрастивно на парах «изображение–текст».
DALL-E	Генерация изображений по тексту. V1 использует дискретный VAE для токенизации изображений и авторегрессионный Transformer над совместным распределением текстовых и графических токенов.
DeBERTa	Decoding-enhanced BERT with disentangled attention. Расцеплённое (disentangled) внимание и mask-декодер, обогащённый информацией об абсолютной позиции слова.
ELECTRA	Малый generator (вроде маленького BERT) заменяет токены через MLM; большой discriminator (основная модель ELECTRA) предсказывает, является ли каждый токен исходным или был заменён.
LLaMA	Decoder-only с пред-нормализацией (RMSNorm), SwiGLU и RoPE.
Mistral	Эффективная decoder-only модель с Grouped-Query Attention (GQA) и Sliding Window Attention (SWA).
RoBERTa	Оптимизированный BERT — больше данных, без NSP, динамическое маскирование, более крупный байтовый BPE-словарь.
T5	Text-to-Text Transfer Transformer. Encoder-decoder, предобученный на задаче восстановления повреждённых спанов (span-corruption).

Модель	Описание
XLNet	Permutation language modeling (моделирование языка на перестановках).
RWKV	Линейное время как у RNN; качество как у трансформеров.
Mamba	Модель пространства состояний (state-space model) — линейная сложность по длине последовательности.

Дополнительная информация

- Encoder-only, encoder-decoder и decoder-only модели:
 - Encoder-only: хорошо подходят для кодирования входа в насыщенное представление; часто применяются для классификации предложений, NER, классификации QA, генерации эмбеддингов. Обычно обучаются через MLM. *Примеры:* BERT.
 - Decoder-only: хорошо подходят для генерации; часто применяются для завершения текста/кода, диалогов, языкового моделирования. Обычно обучаются через Causal Language Modeling. *Примеры:* GPT, LLaMA, Mistral.
 - Encoder-decoder (или seq2seq): хорошо подходят для преобразования текста; часто применяются для машинного перевода, суммаризации, генерации QA, описания изображений. Обычно обучаются через seq2seq-функции потерь. *Примеры:* T5, BART.
- Layer normalization (LayerNorm): нормализует образцы по признаковой размерности, а не признаки по батчам (как в Batch Normalization). Нормализация выполняется по среднему и дисперсии с двумя обучаемыми параметрами: $y = \gamma * (x - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta$. В стандартном Transformer она применялась после multi-head attention и плотного подслоя; в настоящее время чаще используется pre-normalization — до multi-head attention и плотного подслоя.

4.2 BERT

Большая часть информации доступна в [статье о BERT](#).

Ключевые особенности:

- Multi-head attention. Энкодер Transformer. Два размера модели: BASE и LARGE.
- 768-мерное представление используется на всех слоях в BERT Base.
- WordPiece-токенизация, эмбеддинги с словарём в 30 000 токенов. Позиционные эмбеддинги.
- Двунаправленная модель.
- Предобучение через masked language modeling (MLM) — случайно маскируется 15% токенов: 80% заменяются на токен [MASK], 10% — на случайный токен, 10% остаются без изменений — и через next sentence prediction (NSP) — предсказание того, является ли следующее предложение случайным или идёт следом за текущим.

Варианты и расширения

- RoBERTa (Robustly Optimized BERT Approach): без задачи NSP, динамическое маскирование, Byte-Pair Encoding.
- DistilBERT: дистиллированная версия BERT с на 40% меньшим числом параметров, на 60% быстрее при сохранении 97% качества BERT.
- ALBERT (A Lite BERT): факторизованная параметризация эмбеддингов и межслойное разделение параметров, задача на согласованность между предложениями вместо NSP.
- ELECTRA: Replaced Token Detection вместо MLM, архитектура «generator–discriminator» для более эффективного предобучения.
- ModernBERT: обновляет BERT современными улучшениями архитектуры Transformer.

Ссылки

- [Original BERT Paper](#)
- [BERT Explained - Visual Guide](#)
- [Google AI Blog on BERT](#)
- [BERT GitHub Repository](#)

4.3 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) — это гибридный подход, объединяющий генеративные способности LLM с поиском информации в реальном времени из внешних источников знаний. RAG расширяет возможности LLM: сначала из базы знаний или корпуса извлекается релевантная информация, затем она используется для дополнения контекста, передаваемого генеративной модели перед формированием итогового ответа.

RAG призван смягчить ряд ограничений LLM: повысить фактологическую точность, обеспечить указание источников, ослабить проблему ограниченности знаний по дате обучения (knowledge cutoff) и снизить галлюцинации модели.

В оригинальной [статье о RAG](#) использовались предобученный retriever (модель поиска) для извлечения релевантных документов и sequence-to-sequence модель для генерации ответов, обусловленных как запросом, так и извлечёнными документами.

В общем виде разработка RAG-системы предполагает два отдельных процесса: подготовку документов (индексацию) и работу с запросами к ним (поиск и генерацию).

Indexing (офлайн-подготовка данных)

На этом этапе подготавливается внешняя база знаний так, чтобы релевантная информация могла быть эффективно извлечена позднее. Обычно он включает следующие шаги:

- 1. Загрузка данных (Data Loading):** определение и загрузка исходных документов. Они могут поступать из разных источников: файлы документов, базы данных, API, коллаборативные платформы и т.п. Разные источники могут требовать разных подходов к загрузке и обновлению.
- 2. Очистка и предобработка данных (Data Cleaning & Preprocessing):** очистка загруженных данных через удаление нерелевантного содержимого, исправление ошибок и стандартизацию форматов.
- 3. Чанкование документов (Document Chunking / Splitting):** разбиение крупных документов на меньшие управляемые чанки. Это критично, поскольку у LLM ограниченное окно контекста, а поиск работает лучше на сфокусированных фрагментах текста. Каждый чанк может содержать метаданные (исходный документ, дата создания, резюме, теги, ключевые слова, структурная информация). В идеале каждый чанк должен быть достаточно самодостаточным, чтобы быть осмысленным, но при этом достаточно небольшим для эффективного поиска и встраивания в контекст. Распространённые стратегии:
 - Чанкование фиксированного размера (fixed-size chunking): разбиение текста на чанки фиксированного числа символов или токенов, возможно с перекрытием для сохранения непрерывности контекста между чанками.
 - Контентно-зависимое чанкование (content-aware chunking): разбиение по семантическим границам (абзацы, разделы, предложения). Распространённая техника — рекурсивное посимвольное разбиение.
 - Иерархическое чанкование (hierarchical chunking): поддержание родительско-дочерних отношений между чанками (например, раздел → абзац → предложение); позволяет извлекать контекст разной гранулярности.
 - Агентное чанкование (agentic chunking): использование LLM для определения оптимального способа разбиения документов на основе их содержания и структуры.
- 4. Генерация эмбеддингов (Embedding Generation):** использование эмбеддинг-модели для преобразования каждого текстового чанка в эмбеддинг. Эмбеддинг-модели могут быть дообучены на доменных данных для повышения релевантности. Возможно использование нескольких эмбеддинг-моделей или представление одного чанка несколькими векторами (например, отдельный вектор на абзац или отдельные векторы для заголовка и содержимого).
- 5. Индексация (Indexing):** хранение текстовых чанков и их эмбеддингов в эффективном индексе, обычно — векторной базе данных (Pinecone, Weaviate, Chroma, FAISS, Milvus, Qdrant). Это позволяет выполнять быстрый поиск по сходству на основе близости векторов. Метаданные обычно хранятся рядом с эмбеддингами, что позволяет применять фильтрацию до или после векторного поиска.

Retrieval and Generation (онлайн / рантайм-обработка запроса)

Когда пользователь отправляет запрос, система извлекает релевантную информацию и генерирует ответ. Этот этап обычно включает:

- 1. Формулировка/преобразование запроса (Query Formulation/Transformation):** исходный запрос пользователя может быть неоднозначным или лишённым контекста. Уточнение запроса способно улучшить результаты поиска. Возможные подходы:
 - Расширение запроса (Query Expansion): добавление синонимов, связанных терминов, контекста из диалога.
 - Преобразование запроса (Query Transformation): перефразирование запроса, возможно с помощью LLM (Hypothetical Document Embeddings (HyDE), Step-Back Prompting, contextual retrieval).
 - Генерация подзапросов (Sub-Query Generation): разбиение сложного запроса на несколько более простых, результаты которых затем агрегируются.
 - Извлечение метаданных (Metadata extraction): извлечение релевантных метаданных, аналогичных тем, что были получены на этапе подготовки данных.
 - Классификация и маршрутизация запросов (Query classification and routing): направление запросов к специализированным retriever'ам или базам знаний в зависимости от типа запроса (например, поиск по ключевым словам для специфических терминов, векторный поиск — для семантического смысла).
- 2. Эмбеддинг запроса (Query Embedding):** запрос преобразуется в эмбеддинг той же моделью эмбеддингов, что использовалась при индексации.
- 3. Поиск (Search / Retrieval):** эмбеддинг запроса используется для поиска по векторному индексу. Система извлекает k наиболее похожих чанков (top-k результаты) по выбранной метрике сходства (косинусное сходство, скалярное произведение, евклидово расстояние). Может включать:
 - Гибридный поиск (Hybrid Search): сочетание векторного поиска с традиционным поиском по ключевым словам (например, BM25).
 - Фильтрация по метаданным (Metadata Filtering): фильтрация чанков по прикрепленным метаданным до или после векторного поиска.
 - Адаптация к домену / дообучение (Domain Adaptation/Fine-tuning): модели поиска (эмбеддинг-модели) могут дообучаться через контрастивное обучение на доменных парах «запрос–документ» для повышения релевантности.
 - Для улучшения поиска могут использоваться графы знаний.
- 4. Реранкинг (Re-ranking):** шаг реранкинга позволяет повысить точность за счёт использования более качественной модели (cross-encoder или LLM) для повторного упорядочивания top-k чанков. Здесь также может применяться ранжирование с учётом разнообразия (diversity-aware ranking), чтобы итоговый контекст не был слишком избыточным. Другой подход — многоэтапный поиск (multi-stage retrieval): извлечь много → кластеризовать → выполнить реранкинг внутри кластеров → выбрать топ из каждого.
- 5. Augmentation контекста и prompt engineering:** исходный запрос пользователя и итоговый набор извлечённых (и реранжированных) чанков объединяются в единый промпт для LLM. Промпт часто содержит явные инструкции для LLM: разделять запрос и извлечённый контекст, использовать только извлечённую информацию, формировать вывод в требуемом формате и т.п.
- 6. Генерация (Generation):** дополненный промпт передаётся в LLM, которая формирует итоговый ответ. LLM синтезирует информацию из извлечённого контекста, чтобы ответить на запрос пользователя, в идеале опираясь именно на предоставленные фрагменты. Generator может дообучаться через instruction fine-tuning на RAG-задачах; применяются PEFT (для эффективности), RL.
- 7. Постобработка (Post-processing):** фильтрация избыточной информации, модерация для удаления вредоносного содержимого, добавление цитирований.

Дополнительные улучшения и вариации

- Кэширование результатов для часто встречающихся запросов.
- Multi-modal RAG для работы с изображениями, аудио, видео.
- Multi-hop RAG (итеративный поиск) разбивает запрос на подзапросы или выполняет несколько раундов поиска и генерации. Возможно извлечь начальный контекст, использовать LLM для формирования уточняющего запроса на его основе, выполнить поиск снова и т.д., цепочкой собирая доказательства. Это имитирует то, как человек отвечал бы на сложный вопрос, собирая факты пошагово. На практике реализуется через декомпозицию запроса и цепочечный поиск, после чего LLM синтезирует итоговый ответ из всех собранных чанков. Особенно полезно, когда ответ требует информации из нескольких документов.
- Использование router-моделей после запроса и разные модели под разные типы запросов/домены.
- Применимы любые приёмы, применимые к генерации в целом — [SelfRAG](#) для саморефлексии, [chain-of-thought](#), [Agentic Retrieval-Augmented Generation for Time Series Analysis](#) с агентами и др.
- Late-chunking: эмбединг всего документа целиком с последующим чанкованием эмбедингов — эмбединг-модель с длинным контекстом строит токен-эмбединги для каждого токена, после чего они разбиваются и агрегируются (pooling) в чанк-эмбединги.
- RankRAG использует единую LLM как для ранжирования контекстов, так и для генерации ответов, включая в обучение небольшой объём данных о ранжировании.

Оценка качества (Evaluation)

1. Качество поиска (Retrieval Performance):

- **Precision@k**: доля релевантных среди извлечённых документов.
- **Recall@k**: доля всех релевантных документов, которые были извлечены.
- **Mean Average Precision (MAP)**: средняя precision по множеству запросов.
- **NDCG (Normalized Discounted Cumulative Gain)**: измеряет качество ранжирования с учётом релевантности и позиции.
- **Mean Reciprocal Rank (MRR)**: среднее обратных рангов первого релевантного документа.

2. Качество генерации (Generation Performance, условно при заданном поиске):

- **Factual Consistency**: измеряет, согласован ли сгенерированный текст с извлечённым контекстом.
- **ROUGE/BLEU/METEOR**: классические метрики генерации текста.
- **BERTScore/BLEURT**: метрики семантического сходства.
- **QA-based метрики**: использование QA-моделей для проверки фактологических утверждений в сгенерированном тексте.

3. End-to-End качество:

- **RAGAS**: фреймворк для оценки RAG-систем по критериям Faithfulness, Context relevance, Answer relevance.
- **Knowledge Conflicts**: насколько хорошо система обрабатывает противоречивую информацию в извлечённых документах.
- **Human Evaluation** — оценка людьми.

Раздел 5. Тематическое моделирование

5.1 Тематическое моделирование: общая постановка задачи

Тематическое моделирование — это техника обучения без учителя, предназначенная для обнаружения абстрактных тем в коллекции документов, где документ — это любая единица текста, которую проект рассматривает как одну (статья, отзыв, твит, абзац, тикет поддержки). Каждая тема выражается через распределение по словам (классические методы) либо через кластер в пространстве эмбедингов, помеченный представительными словами (методы на основе эмбедингов).

Ключевое отличие от классификации: классификация использует заранее заданные метки, тогда как тематическое моделирование обнаруживает их с нуля. Результатом является структура без учителя, но решение о том, соответствует ли обнаруженная структура задуманной, принимается уже постфактум.

Область развивалась от матричной факторизации (LSA, NMF) через вероятностные порождающие модели (pLSA, LDA) к современным нейросетевым подходам и методам на основе эмбедингов (BERTopic, ETM). Список подходов см. ниже в §5.3. LDA и BERTopic рассматриваются отдельно в силу их значимости.

5.1.1 Ключевые допущения

- Документы представляют собой смеси тем (классические и нейросетевые методы), либо каждый документ принадлежит одному основному кластеру (методы на основе эмбедингов с жёстким назначением)
- Темы — это распределения по словам (классические/нейросетевые) либо кластеры в пространстве эмбедингов, помеченные представительными словами (BERTopic, Top2Vec)
- Bag-of-words — для классических методов; методы на основе эмбедингов наследуют контекстное понимание от своего энкодера

5.1.2 Когда тематическое моделирование не подходит

- Метки уже существуют: классификация с учителем или zero-shot классификация LLM — более прямой путь.
- Требуются стабильные, воспроизводимые метки. Большинство тематических моделей дрейфуют между запусками и между переобучениями; потребители на следующих этапах возражают против такой нестабильности.

- Корпус укладывается в бюджет LLM. Если документы помещаются в контекстное окно или вызовы LLM на каждый документ доступны по цене, проще попросить LLM сразу дать сводки тем. Альтернатива — использовать LLM для генерации синтетических меток на подмножестве и обучить дешёвый классификатор с учителем; это часто превосходит «сырые» темы.
- Корпус мал (меньше нескольких тысяч документов). Эмбединга sentence-transformer'ом и прямой кластеризации обычно достаточно.
- Основная потребность — семантическое сходство, а не интерпретируемые темы. Эмбединги и поиск — правильный инструмент.
- Очень короткие тексты (твиты, запросы, фрагменты чатов). Классические тематические модели плохо работают при такой разреженности. Методы на основе эмбедингов или специализированные модели коротких текстов (Biterm Topic Model) справляются лучше.

5.1.3 Когда тематическое моделирование всё ещё уместно

- Цель — интерпретируемость или открытый разведочный анализ данных.
- Важна дешёвая и/или быстрая инференция. Folding-in у LDA и назначение по ближайшему центроиду у BERTopic стоят миллисекунды на документ; вызовы LLM на каждый документ стоят на порядки больше.
- Распределения по темам нужны как признаки для нижестоящих моделей.
- Результатом должно стать отслеживание распространённости тем во времени. Динамические темы или `topics_over_time()` у BERTopic превращают «корпус плюс временные метки» во временной ряд тем.

5.1.4 Оценка качества

Метрики когерентности

Когерентность измеряет, насколько семантически связаны верхние N слов в каждой теме.

UMass-когерентность: основана на совстречаемости верхних пар слов в документах обучающего корпуса:

$$C_{\text{UMass}} = \frac{2}{N(N-1)} \sum_{i=2}^N \sum_{j=1}^{i-1} \log \frac{D(w_i, w_j) + 1}{D(w_j)}$$

где $D(w_i, w_j)$ — число документов, содержащих оба слова. Значения примерно в диапазоне $[-14, 0]$; чем менее отрицательно, тем лучше.

NPMI (Normalized Pointwise Mutual Information): нормирует PMI к диапазону $[-1, 1]$:

$$\text{NPMI}(w_i, w_j) = \frac{\log \frac{P(w_i, w_j)}{P(w_i)P(w_j)}}{-\log P(w_i, w_j)}$$

Показала лучшую общую корреляцию с человеческими суждениями при оценке классических тематических моделей. Типичные «хорошие» значения — в диапазоне 0.05–0.25 по обзорным исследованиям. Hoyle et al. (2021) показали, что автоматические метрики когерентности слабо коррелируют с человеческими суждениями для нейросетевых тематических моделей; их следует рассматривать как грубую sanity-check, а не как лидерборд.

`C_v`. Составная мера, объединяющая NPMI с косинусным сходством векторов слов в скользящем окне. Диапазон $[0, 1]$. Метрика когерентности по умолчанию в Gensim.

Перплексия

Для вероятностных моделей перплексия измеряет, насколько хорошо модель предсказывает отложенные документы:

$$\text{Perplexity} = \exp\left(-\frac{\sum_{d \in \mathcal{D}} \log p(\mathbf{w}_d)}{N_d}\right)$$

Чем меньше, тем лучше, однако перплексия и человеческие суждения коррелируют отрицательно: модели, оптимизирующие перплексию, склонны давать менее интерпретируемые темы (Chang et al., 2009).

Разнообразие тем

- Topic Uniqueness (TU): доля верхних N слов, появляющихся только в одной теме. $TU = 1$ означает нулевое пересечение слов между темами.
- Доля уникальных слов: процент различных слов среди верхних N списков всех тем.

Ручной осмотр

Автоматические метрики ловят ошибки, но могут одобрить темы, которые человек отверг бы. Что проверять:

- Прочитать верхние 10–20 слов каждой темы.
- Прочитать три–пять представительных документов на тему.
- Спросить себя, не нужно ли слить две темы или, наоборот, разделить одну тему на две.
- Попробовать дать короткий человекочитаемый ярлык; если ярлык не подбирается — тема, скорее всего, шум.

Оценка человеком

- Word intrusion: в верхние слова темы вставляется случайное слово; человек определяет «чужака». Чем выше точность, тем более когерентна тема.
- Topic intrusion: показывают документ с его назначенными темами плюс одну случайную тему; человек определяет лишнюю.

- Прямая оценка: судьи оценивают качество темы по шкале Лайкерта на когерентность, полезность и интерпретируемость.

Визуализация

- pyLDAvis: стандартный интерактивный инструмент для классических моделей (LDA). Проецирует темы в 2D через дивергенцию Джессена —Шеннона и анализ главных координат (`js_PCoA`, опция `mds` по умолчанию) и сравнивает частоты терминов внутри темы с частотами по всему корпусу. См. [демо pyLDAvis](#) как пример скриншота.
- Встроенные визуализации BERTopic: проекции UMAP пространства документов с границами кластеров, intertopic-карты, иерархии тем и темы во времени. См. [галерею визуализаций BERTopic](#).

5.1.5 Выбор числа тем

Для методов, требующих K (LDA, NMF, ProdLDA):

- Запустить несколько значений, построить график когерентности от K , искать пик или изгиб.
- Графики когерентности часто выходят на плато, а не образуют пик: обычно есть несколько разумных значений в зависимости от желаемой детализации.
- Осмотреть темы вручную при кандидатных K . Повторяющиеся ключевые слова между темами сигнализируют о слишком большом числе; слитые темы — о слишком малом.
- Сравнить кандидатные K на train/validation-разбиении, но опираться в основном на когерентность плюс ручной осмотр.
- Стартовый диапазон: $K \in [5, 50]$.

BERTopic и Top2Vec обходят это через HDBSCAN, где `min_cluster_size` косвенно управляет детализацией. Иерархическое слияние позволяет постфактумное укрупнение.

5.1.6 Предобработка

Классические методы (LDA, NMF, LSA):

- Токенизация, приведение к нижнему регистру, удаление пунктуации.
- Удаление стоп-слов с предметно-специфичными списками, а не только с общими (например, «patient» в медицинских корпусах; «thanks», «regards», «http» в e-mail; шаблонные обороты компаний).
- Лемматизация (опционально; повышает интерпретируемость).
- Фильтрация редких и слишком частотных терминов.
- Рассмотреть биграммы/триграммы для многословных понятий.
- NMF и LSA лучше работают с TF-IDF-взвешиванием; LDA использует «сырые» счётчики.

Методы на основе эмбедингов (BERTopic, Top2Vec):

- Минимальная предобработка: трансформер сам обрабатывает семантику.
- Не удалять стоп-слова и не лемматизировать до эмбединга.
- Очистить URL, HTML-теги и спецсимволы.
- Очень короткие документы (<10 слов) дают плохие эмбединги; агрегировать или отфильтровать.

Масштаб

Грубые пороги того, что помещается на одной машине:

- Mallet LDA: одна машина, Java. Тянет миллионы документов при достаточной RAM и времени; лучшее качество тем для классического LDA.
- Gensim LDA: стримит с диска; тянет миллионы документов при константной памяти за счёт online variational Bayes.
- LDA в scikit-learn: загружает корпус целиком в память; практический потолок — около 100 тыс. документов до OOM на типичных машинах.
- BERTopic: основная стоимость — вычисление эмбедингов (GPU-часы на миллион документов с `all-MiniLM-L6-v2`). Память UMAP растёт примерно квадратично; ломается на 1–2 млн документов без `low_memory=True` или приближённых ближайших соседей. HDBSCAN тянет диапазон в миллион документов, если выход UMAP помещается.
- NMF: очень быстр для размеров матриц, с которыми работает; обычно ограничен памятью под матрицу «документ-термин».

Инкрементальные и онлайн-обновления

- Gensim: online variational Bayes — режим обучения по умолчанию; `.update(new_corpus)` подмешивает дополнительные документы без переобучения. Гиперпараметры можно перенастроить или зафиксировать.
- BERTopic: `.partial_fit()` обеспечивает стриминговые обновления порциями документов; `merge_models()` объединяет независимо обученные модели. См. [документацию BERTopic по online topic modeling](#).
- River: стриминговая ML-библиотека с модулем `river.feature_extraction`, поддерживающим инкрементальный TF-IDF и способным питать NMF.
- NMF в scikit-learn: поддерживает `partial_fit` для мини-батч-обновлений через `MiniBatchNMF`.

Дрейф тем и их выравнивание

При переобучении на более новом корпусе ID тем, выдаваемые новой моделью, не соответствуют старым. Распространённые подходы:

- Выравнивание тем через венгерский алгоритм. Вычисляется матрица сходства между распределениями «тема-слово» (или центроидами эмбедингов) старой и новой моделей, далее решается задача оптимального назначения. Библиотеки: `scipy.optimize.linear_sum_assignment`.
- `merge_models` в BERTopic: сливает темы моделей, обученных на непересекающихся данных; также может служить механизмом скользящего обновления, при котором старые темы сохраняются.
- Ручная курация: человеческая ревизия новых тем относительно предыдущих.

Многоязычные корпуса

- BERTopic с многоязычным sentence-энкодером: `paraphrase-multilingual-MiniLM-L12-v2` или `distiluse-base-multilingual-cased-v2` поддерживают 50+ языков и кластеризуют документы между языками, когда цель — единые темы.
- ZeroShotTM / CombinedTM: разработаны для кросс-языкового переноса. Обучаются на одном языке, инферят темы на другом без параллельных корпусов.
- Поязычные классические пайплайны: для LDA/NMF на смешанноязычных корпусах сначала сегментировать по языку, затем обучать отдельные модели на каждом языке. Темы по умолчанию не будут выровнены между языками.

Как на самом деле выглядит выход

- Верхние слова на тему (обычно 10–20): канонический интерпретируемый ярлык.
- Представительные документы на тему: 3–5 экземпляров, делающих тему конкретной для человека.
- Матрица «документ-тема»: либо вероятностная (θ_d из LDA), либо единственный ID кластера (по умолчанию в BERTopic) с опциональным мягким распределением через `.approximate_distribution()`.
- Распространённость тем во времени: при наличии временных меток агрегировать назначения тем по бинам времени. Полезно для трендовых отчётов.
- Куратированные человекочитаемые ярлыки: после ручного осмотра «Тема 3» заменяется на «Жалобы по возвратам и биллингу». Именно это попадает к аналитикам и PM.

Постобработка

- Слить близкие дубликаты тем: графики когерентности не ловят темы, которые говорят одно и то же разными верхними словами. Проверять попарное косинусное сходство по векторам «тема-слово» или центроидам тем.
- Выбросить мусорные темы: шаблонные («thanks, please, regards»), форматные артефакты или темы, ни к одному документу осмысленно не привязанные. Тема -1 BERTopic (выбросы) — крайний случай.
- Переименовать темы: общие c-TF-IDF-ярлыки редко выдерживают обратную связь пользователей. Переписать как короткие именные группы, которые узнает аналитик.
- Осмотреть выбросы: кластер шума может оказаться самым интересным. Документы, которые не вписываются, могут быть ранними сигналами новых тем.

5.1.7 Ссылки

- Roder et al. — Exploring the Space of Topic Coherence Measures (2015)
- Chang et al. — Reading Tea Leaves: How Humans Interpret Topic Models (2009)
- Hoyle et al. — Is Automated Topic Model Evaluation Broken? (2021)
- pyLDavis
- BERTopic visualization gallery

5.2 Latent Dirichlet Allocation (LDA)

Latent Dirichlet Allocation (Blei, Ng, Jordan, 2003) — каноническая вероятностная тематическая модель. Она устраняет переобучение своего предшественника pLSA, накладывая априоры Дирихле как на распределение «документ-тема», так и на распределение «тема-слово». Любая современная тематическая модель до сих пор сравнивается с LDA как с базлайном.

Более широкий контекст (когда LDA уместен по сравнению с другими методами) см. в §5.1.

5.2.1 Порождающий процесс

[!info] Порождающий процесс Порождающий процесс — это рецепт изготовления синтетических данных. LDA постулирует, что каждый документ в корпусе был порождён семплированием из определённой иерархии распределений. Инференция обращает рецепт: по наблюдаемым словам восстанавливаются наиболее вероятные распределения, которые их породили.

[!info] Распределение Дирихле Распределение над распределениями на K -мерном симплексе. Сэмпл из $\text{Dir}(\alpha)$ — это вероятностный вектор $(\theta_1, \dots, \theta_K)$ с $\sum_k \theta_k = 1$. Параметр концентрации α управляет разреженностью: малое α (< 1) концентрирует большую часть массы на нескольких компонентах; большое α (> 1) даёт почти равномерные векторы.

Порождающая история:

1. Для каждой темы $k = 1, \dots, K$: тянем распределение по словам $\phi_k \sim \text{Dir}(\beta)$
2. Для каждого документа d : - Тянем распределение по темам $\theta_d \sim \text{Dir}(\alpha)$ - Для каждой словарной позиции $n = 1, \dots, N_d$:
 - Семплируем тему $z_{d,n} \sim \text{Multinomial}(\theta_d)$

- Сэмплируем слово $w_{d,n} \sim \text{Multinomial}(\phi_{z_{d,n}})$

Структура зависимостей:

```
graph LR
  a(("α")) --> t["θ_d<br/>(doc-topic)"]
  t --> z["z_d,n<br/>(topic)"]
  z --> w["w_d,n<br/>(word)"]
  p["φ_k<br/>(topic-word)"] --> w
  b(("β")) --> p
  classDef prior fill:#f5f5f5,stroke:#999,stroke-dasharray:3 3
  classDef latent fill:#eef,stroke:#447
  classDef observed fill:#efe,stroke:#474
  class a,b prior
  class t,z,p latent
  class w observed
```

Формальная plate-нотация (с D документами, K темами, N_d словами на документ) — стандартный способ изображать это в литературе; каноническую иллюстрацию см. на [странице Wikipedia о LDA](#).

5.2.2 Инференция

Инференция восстанавливает скрытые θ_d и ϕ_k по наблюдаемым словам. Три распространённых подхода:

- Collapsed Gibbs sampling: аналитически интегрирует θ и ϕ , затем итеративно переназначает тему каждому слову, сэмплируя из условного апостериора. Используется в Mallet и `LdaModel` Gensim. Точнее, но медленнее на больших корпусах.
- Вариационный байес: аппроксимирует апостериор tractable-семейством и оптимизирует Evidence Lower Bound (ELBO). Используется в scikit-learn. Быстрее, но аппроксимация может смещать результаты.
- Online Variational Bayes (Hoffman et al., 2010): обрабатывает документы мини-батчами, что обеспечивает стриминговые обновления. Режим по умолчанию для `LdaModel` в свежих версиях Gensim.

Mallet часто предпочитают, когда пользователи хотят collapsed Gibbs sampling и готовы пожертвовать скоростью ради качества тем.

5.2.3 Гиперпараметры

Параметр	Смысл	Значение по умолчанию	Эффект
K	число тем	задаётся вручную	слишком мало → слитые темы; слишком много → дублирующиеся темы
α	концентрация «документ-тема»	$1/K$	низкое (0.01–0.1) → каждый документ концентрируется на немногих темах
β / η /eta/topic_word_prior	концентрация «тема-слово»	$1/V$	низкое → каждая тема концентрируется на немногих характерных словах

Для коротких текстов (твиты, заголовки товаров) дефолтное α даёт почти равномерные распределения по темам и делает модель бесполезной. Установить α ниже дефолта вручную и рассмотреть автонастройку через Gensim (`alpha='auto'`).

5.2.4 Почему счётчики, а не TF-IDF

LDA — это вероятностная порождающая модель над словами, сэмплируемыми из мультиномиальных распределений, то есть математика требует целочисленных счётчиков. TF-IDF-значения — это вещественные веса, поэтому в порождающую историю они не вписываются. Технически использовать TF-IDF с LDA можно, но обычно это даёт темы хуже, чем «сырые» счётчики.

Сравните с NMF, которая является матрично-факторизационной целью без распределённых допущений, поэтому TF-IDF там работает хорошо.

5.2.5 Индуктивный инференс на новых документах

LDA использует folding-in: фиксируются распределения «тема-слово» ϕ_k , выученные на обучении, затем на одном новом документе запускается вариационный вывод (или несколько проходов Гиббса), чтобы вывести его распределение по темам θ_{new} . Сравните с BERTopic, где назначение — это единственный поиск ближайшего центроида после вычисления эмбединга.

5.2.6 Управляемые и supervised-варианты

В реальных проектах часто есть частичные доменные знания: одна тема должна быть про «платежи», другая — про «возвраты», и модель должна это уважать.

- Seeded/Guided LDA: заранее задать несколько слов на тему как якорные. Модель смещается к выдаче тем, содержащих эти якорные слова, продолжая свободно открывать остальные темы. Python-библиотека: `GuidedLDA`.
- CorEx (Correlation Explanation): строго говоря, не LDA, а информационно-теоретическая альтернатива, принимающая якорные слова на тему и часто проще управляемая, чем Seeded LDA. Библиотека: `corextopic`.
- Labeled LDA / SLDA (Supervised LDA): supervised-варианты, обуславливающие темы метками документов. Полезны при частичных метках, когда обнаружение тем должно оставаться внутри каждой помеченной группы.

Для интерпретируемых тем, смещённых по seed-списку, лучшая стартовая точка часто — CorEx; для вероятностной LDA-подобной модели с якорями более прямой выбор — Seeded LDA.

5.2.7 Достоинства и недостатки

Достоинства: теоретически обоснованная, интерпретируемые поддокументные смеси тем, хорошо изученная, дешёвая в обучении и обслуживании по сравнению с методами на основе эмбедингов, зрелая поддержка библиотек.

Недостатки: требуется заранее задавать K , bag-of-words игнорирует порядок слов, результаты варьируются между запусками (гиббсовское семплирование стохастично, поэтому random seed нужно фиксировать ради воспроизводимости), чувствительна к предобработке, плохо работает на коротких текстах, не способна улавливать семантическое сходство (для LDA «car» и «automobile» — два несвязанных токена).

5.2.8 Когда LDA всё ещё уместен

- Сценарий требует явных вероятностных поддокументных смесей тем, а не жёстких назначений в кластер.
- Документы средней длины и хорошо оформленные (новостные статьи, статьи, отчёты).
- Приоритет — модель, дешёвая в обучении и обслуживании, с предсказуемой стоимостью инференции на документ.
- Уже работает выстроенный пайплайн на LDA, и смена инструментов обойдётся дороже, чем улучшение существующей модели.
- Распределения по темам нужны как признаки для нижестоящего классификатора: векторы θ_d из LDA — естественный вход.

Если LDA не справляется, следующие шаги: NMF (для более чистых тем на коротком/шумном тексте), BERTopic (для семантического сходства) или ETM (для больших словарей с редкими словами).

```
[!example]- Пример кода (gensim, включая folding-in для новых документов) ```python from gensim import corpora from gensim.models import LdaMulticore from gensim.parsing.preprocessing import preprocess_string
```

--- Train ---

```
docs = [preprocess_string(d) for d in train_corpus] dictionary = corpora.Dictionary(docs) dictionary.filter_extremes(no_below=5, no_above=0.8) corpus = [dictionary.doc2bow(doc) for doc in docs]

lda = LdaMulticore( corpus=corpus, id2word=dictionary, num_topics=20, alpha='symmetric', eta='auto', passes=10, random_state=42, )
```


Top words per topic

```
for i, topic in lda.print_topics(num_words=10): print(f"Topic {i}: {topic}")
```

--- Folding-in: infer topics for a NEW document without retraining ---

```
new_doc = preprocess_string("The central bank raised rates again.") new_bow = dictionary.doc2bow(new_doc) new_topics = lda[new_bow] # list of (topic_id, probability) print(new_topics) ""
```

5.2.9 Ссылки

- Blei, Ng, Jordan — Latent Dirichlet Allocation (2003)
- Hoffman, Blei, Bach — Online Learning for Latent Dirichlet Allocation (2010)
- Gensim LDA documentation
- GuidedLDA — seeded LDA implementation
- CorEx topic modeling library
- Mallet — Java toolkit, Gibbs-sampling LDA
- Wikipedia — Latent Dirichlet Allocation (plate notation figure)

5.3 Методы тематического моделирования (обзор семейств методов)

Обзор основных методов тематического моделирования, упорядоченных примерно по исторической последовательности появления (матричная факторизация → вероятностные порождающие модели → нейросетевые → на основе эмбедингов). О том, когда применять тематическое моделирование, как оценивать результаты и о других вопросах, см. §5.1.

5.3.1 Классические методы

Latent Semantic Analysis (LSA / LSI)

Deerwester et al., 1990. Самый ранний подход, основанный на снижении размерности через сингулярное разложение (SVD).

Имея TF-IDF-взвешенную матрицу «документ-термин» X :

$$X_{m \times n} = U_{m \times K} \Sigma_{K \times K} V_{K \times n}^T$$

где U улавливает связи «термин-тема», Σ содержит сингулярные значения, V улавливает связи «документ-тема», а K — число оставленных размерностей.

Усечение до верхних K сингулярных значений вынуждает синонимы кластеризоваться вместе и частично решает полисемию, проецируя термины и документы в общее латентное пространство.

[!info] Полисемия Полисемия — это когда одно слово несёт несколько значений (bank = берег реки / финансовое учреждение; apple = фрукт / компания). Bag-of-words-методы трактуют каждое вхождение слова как один и тот же токен, поэтому не способны различать эти смыслы по контексту. Методы на основе эмбедингов справляются с полисемией естественно, потому что контекстные энкодеры выдают разные векторы для одного и того же слова в разных контекстах.

Достоинства: детерминирован, быстр, улавливает паттерны встречаемости. Недостатки: размерности тем могут иметь отрицательные веса (тяжело интерпретировать как вероятности), нет вероятностного основания, требуется задавать K .

Probabilistic LSA (pLSA)

Hofmann, 1999. Добавляет вероятностную интерпретацию к LSA:

$$P(w \mid d) = \sum_{k=1}^K P(w \mid z_k) P(z_k \mid d)$$

где z_k — латентная тема. Обучается через EM-алгоритм. pLSA исторически наводит мост между LSA и LDA: ввела идею «документ как смесь тем», но не имеет априоров на распределения «документ-тема», поэтому число параметров растёт линейно с размером корпуса, и модель не может приписывать вероятности невиданным документам.

Latent Dirichlet Allocation (LDA)

Blei, Ng, Jordan, 2003. Каноническая вероятностная тематическая модель. Чинит переобучение pLSA, добавляя априоры Дирихле к распределениям «документ-тема» и «тема-слово». Распределение тем каждого документа θ_d и распределение слов каждой темы ϕ_k выводятся из наблюдаемых слов через гиббсовское семплирование или вариационный вывод.

Подробнее см. §5.2.

Non-negative Matrix Factorization (NMF)

Lee & Seung, 1999. Раскладывает матрицу «документ-термин» V на две неотрицательные матрицы:

$$V_{m \times n} \approx W_{m \times K} H_{K \times n}, \quad \min_{W, H \geq 0} \|V - WH\|_F^2$$

где W улавливает веса «документ-тема», а H — веса «тема-слово». Это соответствует соглашению sklearn (`components_` — это матрица «тема-слово» H); оригинальная статья Lee & Seung использует транспонированное соглашение (V как «термин-документ», W как «термин-тема», H как «тема-документ»). Ограничение неотрицательности даёт естественно разреженные, аддитивные, частично-частные представления.

NMF обычно использует TF-IDF-вход (в отличие от LDA, которому нужны целочисленные счётчики из-за её мультиномиального порождающего предположения). TF-IDF понижает вес общих высокочастотных слов, что обычно даёт более чистые темы.

Подсказка по решателю scikit-learn: использовать `solver='cd'` (coordinate descent) с дефолтной потерей Фробениуса для скорости. Переключиться на `solver='mu'` (multiplicative update), если оптимизируется по дивергенции Кульбака–Лейблера (`beta_loss='kullback-Leibler'`), что даёт более LDA-подобные вероятностные темы.

Достоинства: быстра, детерминирована при заданной инициализации, чистые разреженные темы, хорошо работает с TF-IDF. Недостатки: требуется задавать K , нет вероятностной интерпретации, результаты зависят от инициализации.

5.3.2 Нейросетевые тематические модели

ProdLDA

Srivastava & Sutton, 2017. Первый эффективный подход с нейросетевым вариационным выводом для LDA. Использует архитектуру вариационного автоэнкодера (VAE): энкодер отображает bag-of-words-вход в пропорции тем, декодер реконструирует документ.

Ключевая инновация — аппроксимация априора Дирихле логистически-нормальным распределением, что обходит проблему «коллапса компонент», при которой ранние нейросетевые подходы выдавали вырожденные темы. Обучается максимизацией Evidence Lower Bound (ELBO). Намного быстрее инференция, чем у гиббсовского семплирования, и обрабатывает новые документы без переобучения.

Embedded Topic Model (ETM)

Dieng, Ruiz, Blei, 2020. Комбинирует тематическое моделирование с эмбедингами слов. Каждое слово и каждая тема живут в одном эмбединговом пространстве; вероятность слова w при теме k :

$$P(w | k) \propto \exp(\mathbf{e}_w^\top \mathbf{t}_k)$$

где $\mathbf{e}_w$ — эмбединг слова, а $\mathbf{t}_k$ — эмбединг темы. Можно использовать предобученные эмбединги (Word2Vec, GloVe) или обучать совместно. Лучше стандартного LDA справляется с большими словарями и редкими словами, потому что семантически близкие слова делят эмбединговую структуру.

Contextualized Topic Models (CombinedTM, ZeroShotTM)

Bianchi et al., 2021. Расширяет ProdLDA, подавая в энкодер эмбединги sentence-transformer'ов наряду с (или вместо) bag-of-words-представлениями. Два варианта:

- CombinedTM: BoW + sentence-эмбединги. Обычно показывает более высокую когерентность из двух на стандартных бенчмарках.
- ZeroShotTM: только sentence-эмбединги. Позволяет кросс-языковое тематическое моделирование: обучается на английском, инферит темы на немецком без параллельных корпусов.

5.3.3 Методы на основе эмбедингов

BERTopic

Grootendorst, 2022. Модульный пайплайн: эмбединг документов через sentence-transformers, снижение размерности через UMAP, кластеризация через HDBSCAN и пометка кластеров через class-based TF-IDF. Автоматически определяет число тем и поддерживает динамический, иерархический, управляемый и онлайн-режимы.

Подробнее см. §5.4.

Top2Vec

Angelov, 2020. Совместно эмбедингирует документы и слова, применяет UMAP + HDBSCAN для нахождения плотных кластеров. Векторы тем — центроиды кластеров; слова темы — ближайшие к центроиду слова в эмбединговом пространстве. Подобная философия с BERTopic, предшествовал ему, использует эмбединговое расстояние, а не c-TF-IDF, для представления тем.

5.3.4 Обнаружение тем с помощью LLM

Практичный гибрид, хорошо работающий на практике: запустить BERTopic для кластеризации, затем использовать LLM как модель представления для пометки каждого кластера. Это даёт масштабируемость кластеризации на эмбедингах с человекочитаемыми ярлыками от LLM без оплаты вызовов LLM на каждый документ.

Полностью LLM-основанные подходы (TopicGPT) используют LLM для генерации ярлыков тем прямо из выборок документов, опционально в иерархическом цикле уточнения. Ярлыки сразу читаемы, а детализацию можно задавать на естественном языке, но стоимость масштабируется с размером корпуса и результаты недетерминированы.

5.3.5 Варианты для коротких текстов

Biterm Topic Model (BTM)

Yan et al., 2013. Разработан для очень коротких текстов (твиты, запросы, заголовки), где классический LDA борется с внутридокументной разреженностью. Вместо моделирования документов BTM моделирует неупорядоченные пары слов (бигермы), извлечённые из корпуса целиком, и трактует весь корпус как одну неявную смесь тем. Каждый бигерм порождается семплированием темы из распределения тем уровня корпуса, затем оба слова семплируются независимо из распределения слов этой темы.

Меняет поддокументные пропорции тем на лучшее качество тем на коротком тексте. Доступен в оригинальной C++-реализации и Python-портах (bitermplus, biterm).

5.3.6 Другие заметные подходы

BigARTM

Vorontsov & Potapenko, 2015. Расширяет pLSA аддитивной регуляризацией (ARTM): несколько регуляризаторов (разреженность, декорреляция, иерархия, надзорные метки) объединяются в одну целевую функцию:

$$L(\Phi, \Theta) + \sum_i \tau_i R_i(\Phi, \Theta) \rightarrow \max$$

Эта гибкость позволяет одновременно оптимизировать разреженность тем, различимость между темами и включать побочную информацию.

Dynamic Topic Models

Blei & Lafferty, 2006. Расширяет LDA на временно упорядоченные корпуса. Распределения тем эволюционируют во времени через гауссов шум:

$$\alpha_t \mid \alpha_{t-1} \sim \mathcal{N}(\alpha_{t-1}, \sigma^2 I)$$

Улавливает, как темы меняют значение со временем. Доступна в Gensim (DtmModel) и tomatopy. .topics_over_time() BERTopic даёт более простую альтернативу за счёт пересчёта c-TF-IDF на каждом временном бине, хотя это не настоящая порождающая модель.

Иерархические и непараметрические варианты

- Hierarchical LDA (hLDA). Открывает древовидную иерархию тем с помощью nested Chinese Restaurant Process. Не требует задавать *K* или глубину дерева.
- Hierarchical Dirichlet Process (HDP). Непараметрическое расширение LDA, выводящее *K* из данных. Доступна в Gensim и tomatopy.
- Correlated Topic Model (CTM). Заменяет априор Дирихле на логистически-нормальный, позволяя темам коррелировать (например, «генетика» и «биология» встречаются вместе чаще, чем «генетика» и «кулинария»).
- Structural Topic Model (STM): включает метаданные уровня документа (дата, источник, автор) как ковариаты, влияющие на распространённость и содержание тем. Полезна, когда темы не независимы от атрибутов документа.

5.3.7 Когда что использовать

Сценарий	Рекомендация
Короткие тексты (твиты, отзывы)	BERTopic, Top2Vec
Длинные документы (статьи)	LDA, NMF, BERTopic
Смешанное членство (документ в нескольких темах)	LDA, NMF, CombinedTM
Автоматическое число тем	BERTopic, Top2Vec, HDP
Ограниченные ресурсы / быстрый результат	NMF (самый быстрый), LDA
Наивысшая отчётная когерентность на стандартных бенчмарках	BERTopic, CombinedTM
Темпоральная эволюция тем	BERTopic .topics_over_time(), DTM
Кросс-языковые темы	ZeroShotTM / CombinedTM
Большой словарь с редкими словами	ETM
Иерархическая структура тем	hLDA, BERTopic (иерархический режим)
Максимальная гибкость через регуляризацию	BigARTM
Темы с учётом метаданных (дата, автор, источник)	STM
Инженерия признаков для нижестоящего классификатора	LDA (вероятностный θ_d), NMF
LLM-качество ярлыков на масштабе	BERTopic с LLM-моделью представления
Разовая сводка тем, малый корпус	Промптить LLM напрямую

BERTopic — распространённая стартовая точка для новых проектов в текущей практике. LDA и NMF остаются актуальными для условий с ограничениями по ресурсам, требований к смешанному членству и сложившихся пайплайнов.

5.3.8 Инструменты и библиотеки

В таблице ниже `CTM` означает Correlated Topic Model (с логистически-нормальным априором); `CombinedTM` и `ZeroShotTM` — это Contextualized Topic Models. `DTM` — Dynamic Topic Model, `SLDA` — Supervised LDA, `LLDA` — Labeled LDA.

Библиотека	Методы	Примечания
Gensim	LDA, LSA/LSI, HDP, LdaMallet wrapper	Память-эффективный стриминг; лучший выбор для классических методов
scikit-learn	NMF, LDA, TruncatedSVD (LSA)	Плотная интеграция с ML-пайплайнами; in-memory потолок
BERTopic	BERTopic (динамический, иерархический, управляемый, онлайн)	Модульная; поддерживает настройку представлений через LLM
Top2Vec	Top2Vec	Простой API; автоматический K
tomotopy	LDA, HDP, CTM, DTM, SLDA, LLDA	Очень быстрый C++-бэкенд; много вариантов LDA
OCTIS	LDA, NMF, ETM, ProdLDA, CTM и др.	Фреймворк для бенчмаркинга с оптимизацией гиперпараметров
contextualized-topic-models	CombinedTM, ZeroShotTM	Поддержка кросс-языковости
Mallet	LDA (Gibbs sampling)	Часто даёт лучшие результаты LDA; у Gensim есть обёртка

```
[!example]- Пример кода (scikit-learn NMF & LDA, BERTopic)
python from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.decomposition import NMF, LatentDirichletAllocation from sklearn.datasets import fetch_20newsgroups

data, _ = fetch_20newsgroups(shuffle=True, random_state=1, remove=('headers', 'footers', 'quotes'), return_X_y=True) docs = data[:2000] n_topics = 10
```

--- NMF with TF-IDF ---

```
tfidf = TfidfVectorizer(max_df=0.95, min_df=2, stop_words='english') X_tfidf = tfidf.fit_transform(docs) nmf = NMF(n_components=n_topics, random_state=1, init='nndsvd').fit(X_tfidf)
```

--- LDA with raw counts ---

```
tf = CountVectorizer(max_df=0.95, min_df=2, stop_words='english') X_tf = tf.fit_transform(docs) lda = LatentDirichletAllocation(n_components=n_topics, random_state=0).fit(X_tf)

def show_topics(model, feature_names, n_words=10):
    for i, topic in enumerate(model.components_):
        words = [feature_names[j] for j in topic.argsort()[: -n_words - 1: -1]]
        print(f"Topic {i}: {' '.join(words)}")

show_topics(nmf, tfidf.get_feature_names_out()) show_topics(lda, tf.get_feature_names_out())
```

--- BERTopic ---

```
from bertopic import BERTopic topic_model = BERTopic() topics, probs = topic_model.fit_transform(docs) topic_model.get_topic_info()
```

5.3.9 Ссылки

- Blei, Ng, Jordan — Latent Dirichlet Allocation (2003)
- Grootendorst — BERTopic: Neural topic modeling with a class-based TF-IDF procedure (2022)
- Blei's Topic Modeling page
- BERTopic documentation
- Gensim topic modeling documentation
- OCTIS — Comparing Topic Models
- scikit-learn Topic Extraction with NMF and LDA
- Structural Topic Model
- BigARTM documentation

5.4 BERTopic

BERTopic — это модульный пайплайн тематического моделирования. Вместо того чтобы быть одной порождающей моделью, он компонует четыре независимо заменяемых шага: эмбединг, снижение размерности, кластеризация и пометка. Шаг пометки использует class-based-вариант TF-IDF, введённый специально для тем, полученных из кластеров.

Более широкий контекст (когда BERTopic уместен по сравнению с LDA, NMF или просто кластеризацией) см. в §5.1.

5.4.1 Пайплайн

```
graph LR
    D[Documents] --> E[Sentence<br/>Transformer]
    E --> U[UMAP<br/>dim reduction]
    U --> H[HDBSCAN<br/>clustering]
    H --> C[c-TF-IDF<br/>labeling]
    C --> T[Topics +<br/>top words]
    R[Representation<br/>model<br/>KeyBERT / MMR / LLM] -.optional.-> C
```

1. Эмбединг документов. По умолчанию: `all-MiniLM-L6-v2` через `sentence-transformers`. Допустима любая эмбединговая модель: доменно-специфические энкодеры часто превосходят универсальные.
2. Снижение размерности. UMAP (Uniform Manifold Approximation and Projection) обычно используется здесь, потому что склонен сохранять соседскую структуру, полезную для кластеризации. Типичная цель: 5–10 размерностей.
3. Кластеризация через HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise). Density-based; не требует K ; назначает документы-выбросы кластеру `-1` вместо насильственного вписывания в тему.
4. Извлечение слов темы через c-TF-IDF (см. ниже).
5. (Опционально) Тонкая настройка представлений с помощью модели представления (`KeyBERTInspired`, `MaximalMarginalRelevance` или LLM) для замены выхода c-TF-IDF на более описательные ярлыки.

Каждый шаг — аргумент конструктора, поэтому любую эмбединговую модель, редуктор, кластеризатор или векторизатор можно заменить, не переписывая остальное. Практики часто ссылаются на эту модульность, сравнивая BERTopic с ранними нейросетевыми тематическими моделями.

5.4.2 c-TF-IDF

Class-based TF-IDF трактует каждый кластер как один сконкатенированный документ, затем считает TF-IDF между кластерами. Версия, сейчас реализованная в BERTopic (с конца 2022 года), использует аддитивное сглаживание внутри логарифма:

$$\text{c-TF-IDF}(t, c) = \frac{tf(t, c)}{\sum_i tf(t, i)} \cdot \log\left(1 + \frac{A}{\sum_i tf(i, i)}\right)$$

где:

- $tf_{t,c}$: частота термина t в кластере c
- $\sum_i tf_{t,i}$: всего слов в кластере c (L1-нормализация вектора term-frequency)
- A : среднее число слов на кластер
- f_t : суммарная частота термина t по всем кластерам

Текущая реализация использует аддитивное сглаживание в IDF-подобной части. Результат: слова, частые в этом кластере и относительно редкие по всем кластерам.

[!note] Оригинальная статья 2022 года и более ранние версии библиотеки использовали $\log(m / \sum_i tf_{t,i})$, где m — число классов. Старые упоминания, цитирующие эту формулу, описывают ту же идею, просто без сглаживания, которое применяет текущая библиотека.

BERTopic также поддерживает BM25-взвешенный вариант через `bm25_weighting=True` в `ClassTfidfTransformer`, что может помочь при сильно неравном размере классов. Отдельная опция `reduce_frequent_words=True` применяет сублинейное масштабирование TF (`sqrt(tf)` вместо `tf`), что подавляет эффект корпусно-общих высокочастотных терминов.

5.4.3 Ключевые возможности

- Динамические темы (`.topics_over_time()`): пересчитывает c-TF-IDF по бидам времени; проще, чем настоящая Dynamic Topic Model, но не порождающая модель.
- Иерархическое слияние тем: постфактумное агломеративное слияние на основе сходства тем; позволяет исследовать детализацию без переобучения.
- Guided BERTopic: засеивание темы списком представительных слов; пайплайн смещает кластеры в сторону этих сидов.
- Онлайн / инкрементальный режим: `.partial_fit` поддерживает стриминговые обновления; `merge_models` объединяет модели, обученные на разных подмножествах.
- Мягкое назначение: `.approximate_distribution()` возвращает поддокументные вероятности тем вместо единственного жёсткого ID кластера. Полезно для выходов в стиле «смешанное членство».
- LLM-основанные модели представления: передать OpenAI или LangChain -модель представления, чтобы заменить c-TF-IDF-ярлыки на сгенерированные LLM. См. §5.3.4.

Визуальные примеры этих возможностей разбирает [документация по визуализации BERTopic](#): `.visualize_topics()` (intertopic distance map), `.visualize_hierarchy()` и `.visualize_topics_over_time()`.

5.4.4 Режимы отказа

- UMAP стохастичен, поэтому разные запуски дают разное число тем. Фиксировать `random_state` для воспроизводимости.
- Шумовой кластер HDBSCAN может поглощать слишком много документов. Распространённые корректировки: понизить `min_cluster_size` и/или `min_samples`.
- Темы наследуют смещения эмбединговой модели. Митигация: переключиться на доменно-релевантный энкодер, дообучить энкодер на целевом корпусе или использовать задаче-специфические кодировки.

5.4.5 Индуктивный инференс на новых документах

`transform()` BERTopic назначает тему новому документу в три шага: эмбединтит документ обученным энкодером, проецирует через обученный UMAP (через `transform`, не `fit_transform`), затем вызывает `approximate_predict` HDBSCAN'a, чтобы получить ближайший кластер. HDBSCAN использует сжатое дерево кластеров с обучения: это не метод на центроидах, поэтому назначение отражает структуру плотности, а не расстояние до среднего по кластеру. Для мягкого назначения `.approximate_distribution()` возвращает поддокументный вектор вероятностей тем через скользящее окно по токенам, независимо от модели HDBSCAN. Любой из путей обычно на один-два порядка быстрее `fold-in` у LDA на этапе инференции.

5.4.6 Смешанное членство

По умолчанию BERTopic возвращает одну тему на документ. Это менее естественно, чем LDA, когда документ действительно смешивает несколько тем сразу. `.approximate_distribution(docs)` возвращает поддокументный вектор вероятностей тем, вычисляемый через скользящее окно по токенам. Это постфактумная аппроксимация, а не вероятностная модель, но её обычно хватает, чтобы выявить документы с осмысленными вторичными темами.

5.4.7 Guided BERTopic

Когда известно, какие темы должны существовать заранее, передаётся список `seed`-списков слов:

```
seed_topic_list = [
    ["loan", "credit", "mortgage", "interest"],
    ["team", "game", "match", "score"],
]
topic_model = BERTopic(seed_topic_list=seed_topic_list)
```

Пайплайн смещает векторизатор и вычисление c-TF-IDF в сторону этих сидов, повышая шанс, что появится тема, содержащая эти якорные слова. Прочие темы открываются свободно. Полезно, когда доменные знания частичны и проекту нужно гарантированное покрытие известных категорий без полного supervision.

5.4.8 Достоинства и недостатки

Достоинства: минимальная предобработка (трансформер обрабатывает семантику), автоматическое K через HDBSCAN, много возможностей (динамический / иерархический / управляемый / онлайн-режимы в одной библиотеке).

Недостатки: жёсткая кластеризация по умолчанию (мягкое назначение требует дополнительного шага `.approximate_distribution()`), вычислительно тяжелее классических методов, память UMAP растёт примерно квадратично, качество эмбедингов — узкое место в специализированных доменах.

```
[!example]- Пример кода (BERTopic с LLM-ярлыками тем)
"""python from bertopic import BERTopic from bertopic.representation import OpenAI,
KeyBERTInspired from sentence_transformers import SentenceTransformer from sklearn.datasets import fetch_20newsgroups import openai

docs = fetch_20newsgroups( subset="all", remove=("headers", "footers", "quotes"))["data"]

embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
```

Option 1: lightweight representation tuning via KeyBERT


```
representation_model = KeyBERTInspired()
```

Option 2: LLM-generated labels (uncomment to use)

```
client = openai.OpenAI()
```

```
representation_model = OpenAI(client, model="gpt-4o-mini",  
chat=True)
```

```
topic_model = BERTopic( embedding_model=embedding_model, representation_model=representation_model, min_topic_size=20,  
random_state=42, verbose=True, ) topics, probs = topic_model.fit_transform(docs)
```

Inspect top topics

```
print(topic_model.get_topic_info().head(10))
```

Soft assignment for mixed-membership-style outputs

```
topic_distr, _ = topic_model.approximate_distribution(docs[:100])
```

Inference on new documents (no retraining)

```
new_docs = ["The central bank raised rates again."] new_topics, _ = topic_model.transform(new_docs) ""
```

5.4.9 Ссылки

- Grootendorst — BERTopic: Neural topic modeling with a class-based TF-IDF procedure (2022)
- BERTopic documentation
- BERTopic c-TF-IDF source code
- BERTopic visualization gallery
- BERTopic guided topic modeling guide
- sentence-transformers library
- UMAP documentation
- HDBSCAN documentation